

Modern Tooling and DevOps Integration



MONITORING

Monitoring provides continuous visibility into the health and performance of systems, applications, and infrastructure.

Teams use monitoring to detect issues early, reduce downtime, and ensure services meet business and operational objectives.

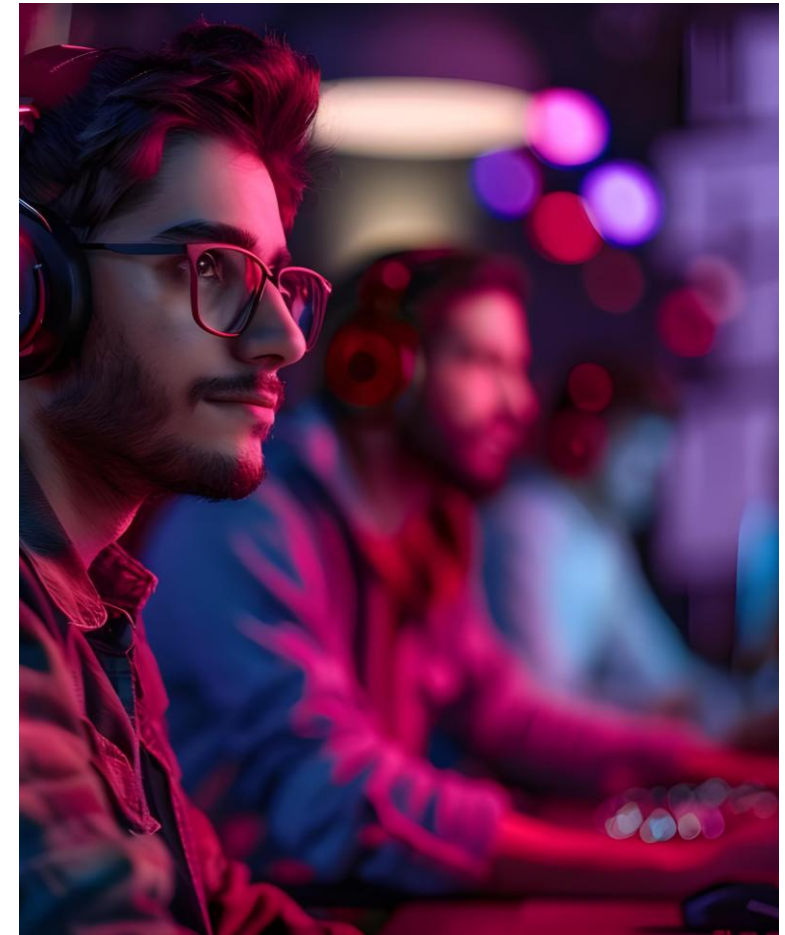
Takeaway: Effective monitoring helps teams identify problems quickly and maintain reliable services.



WHY MONITOR

Monitoring supports many operational and business monitoring scenarios. It should be flexible enough for simple and complex environments.

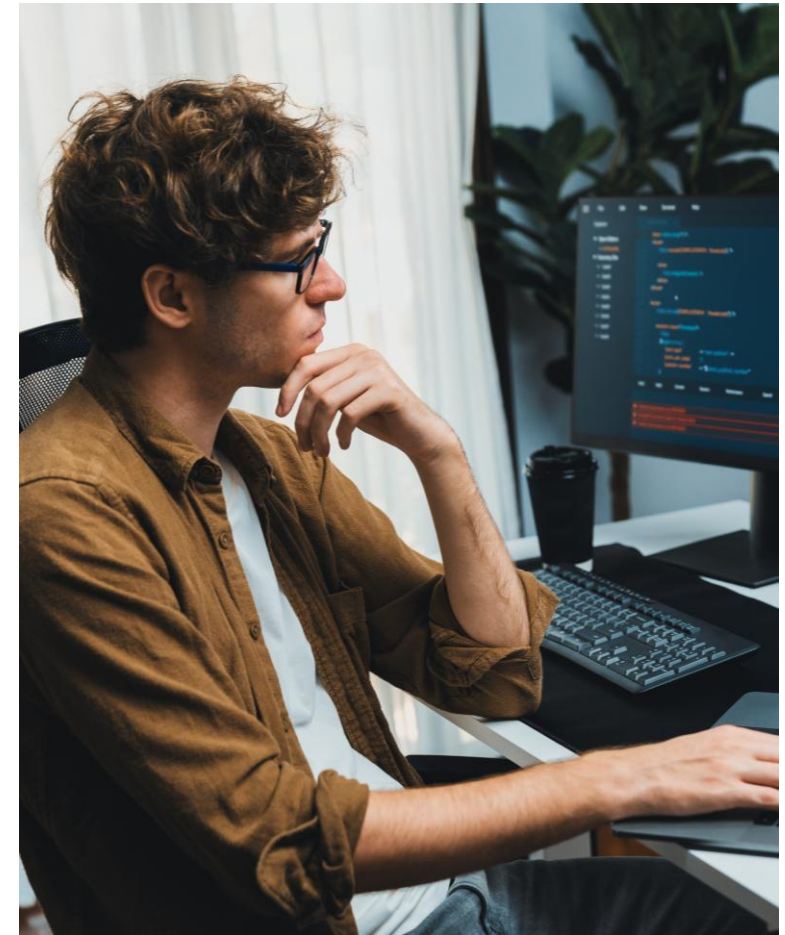
- Response time tracking
- Error rate monitoring
- Capacity planning
- Incident investigation
- SLOs/SLAs
- Alerts



WHY MONITORING MATTERS

Anything that moves in production should be monitored. Without monitoring, reliability cannot be measured or improved.

- Applications
- Infrastructure
- Networks
- Dependencies



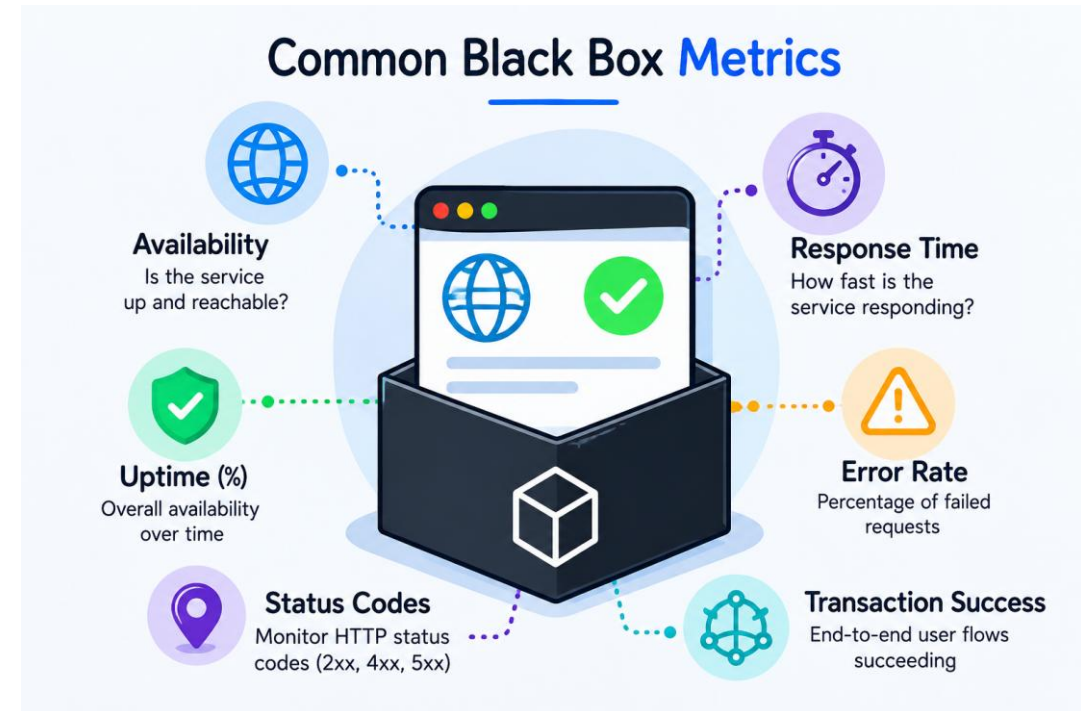
BLACK BOX MONITORING

Tests a system from the outside, just like a real user. It focuses on whether a service is available and responding correctly.

Examples

- Website availability checks
- API health checks
- Login transaction tests

Takeaway: Black box monitoring validates the user experience without needing access to internal components.



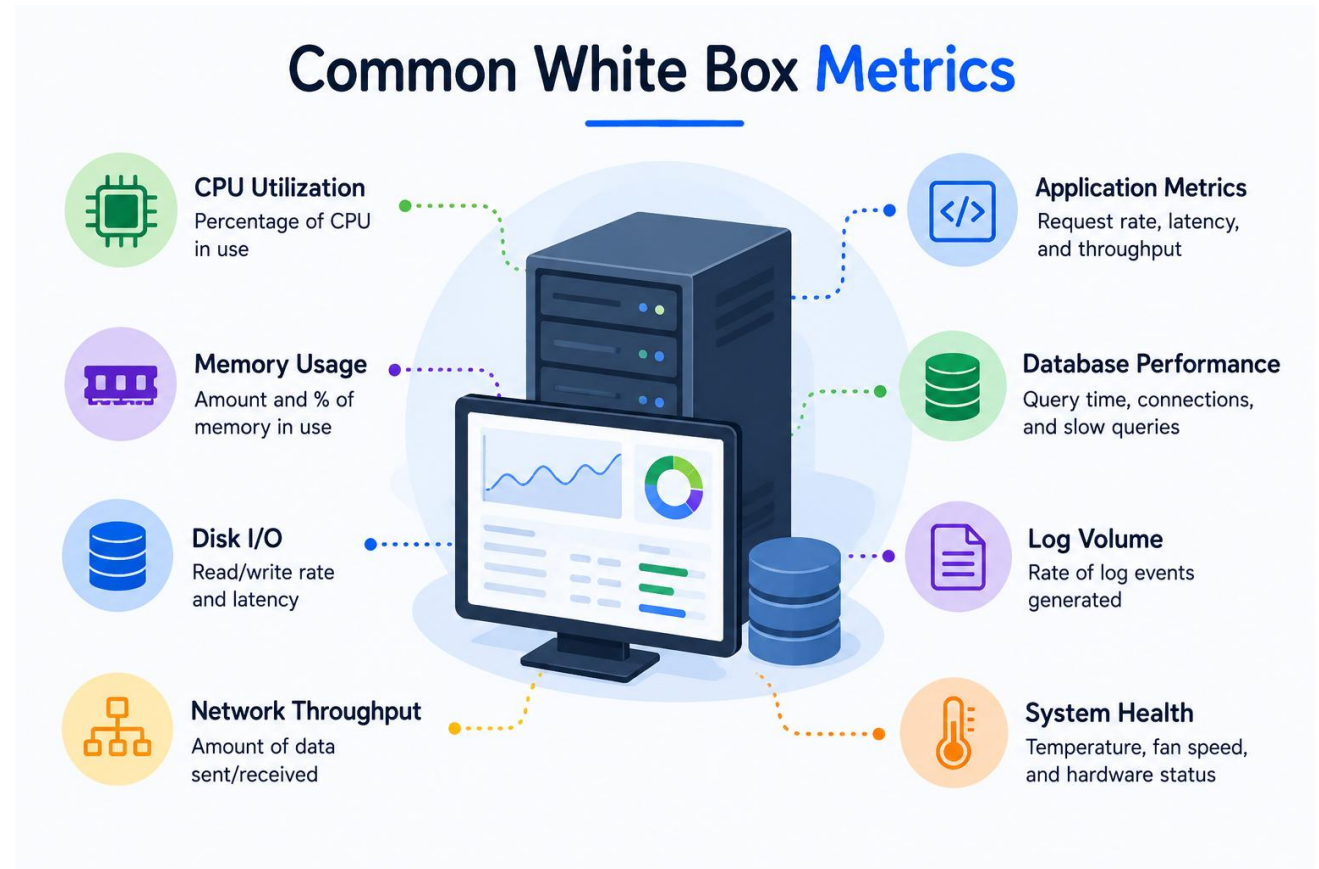
WHITE BOX MONITORING

Collects data from inside a system to understand how it is operating.

Examples

- CPU and memory usage
- Application metrics
- Database performance

Takeaway: White box monitoring provides visibility into the internal health of applications and infrastructure.



MONITORING VS OBSERVABILITY

Monitoring tells you when a known problem occurs. Observability helps you understand unknown failures.

- Monitoring is threshold-based
- Observability is exploratory
- Dashboards vs investigations
- Both are required

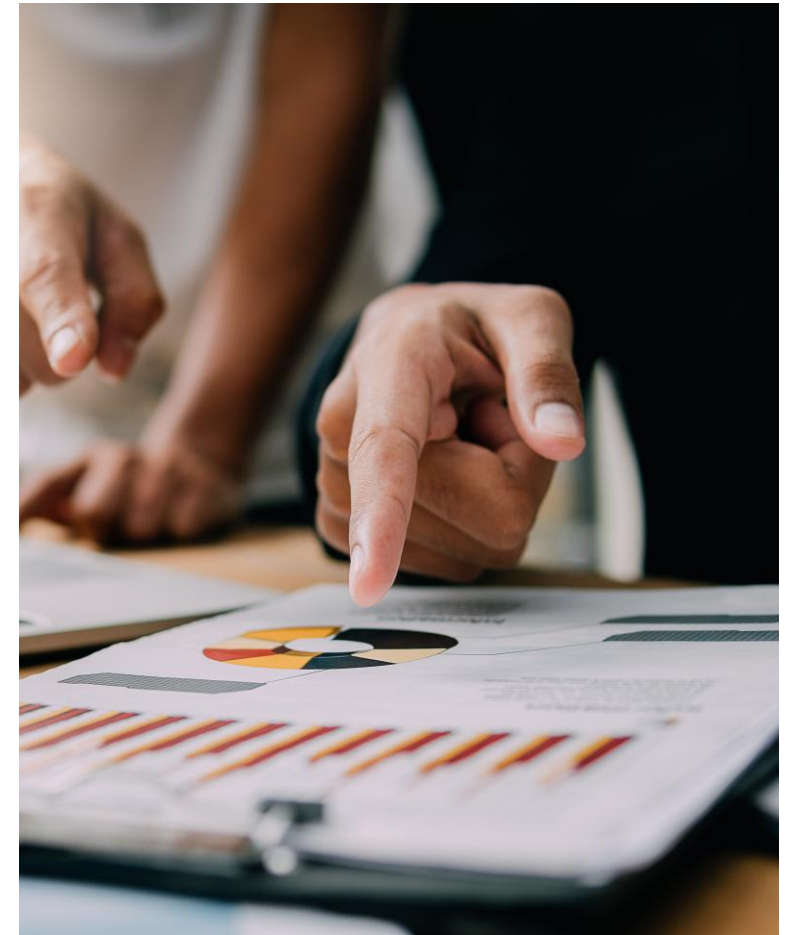


SERVICE LEVEL AGREEMENTS (SLA)

An SLA is a formal agreement between a service provider and a customer that defines expected service levels. If agreed targets are not met, the provider may owe service credits, refunds, or other penalties.

Examples

- 99.9% uptime
- API responses under 500ms
- 95% of tickets resolved within 24 hours

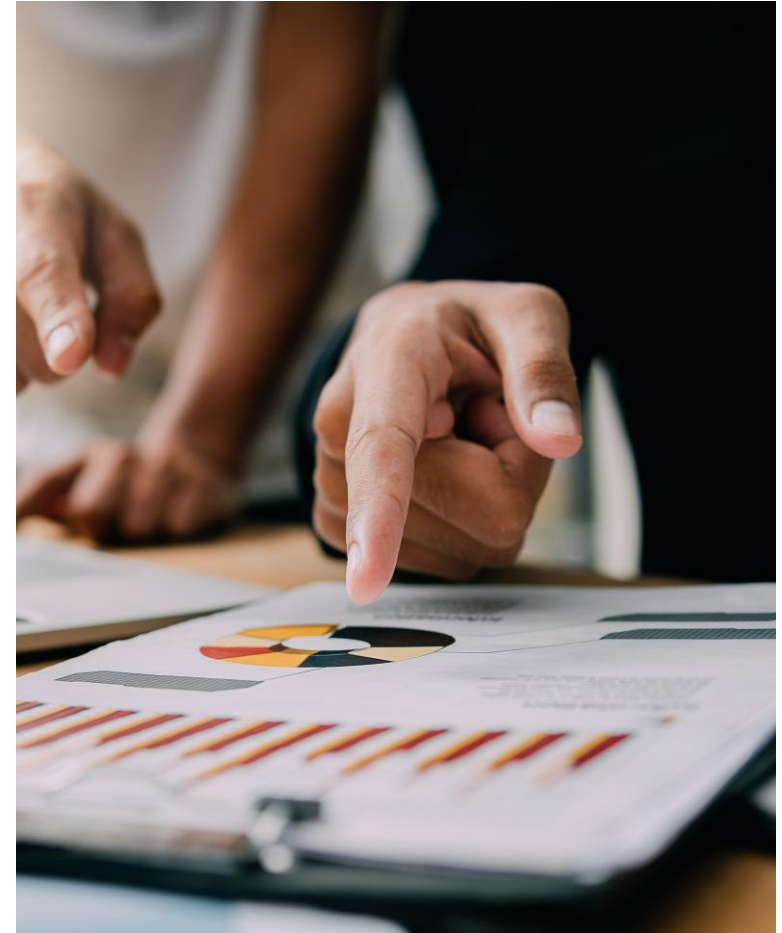


SERVICE LEVEL AGREEMENTS (SLA)

Key Points

- Customer-facing commitment
- Defines measurable targets, typically with Black Box Metrics
- May include penalties for violations

Takeaway: An SLA is a promise to customers about the level of service they can expect.



SLOS AND SLAS

Service reliability is defined using measurable objectives. Monitoring provides the data needed to track them.

- SLOs define internal targets
- SLAs define customer promises
- Metrics validate compliance
- No data means no guarantees



MONITORING SERVICE RELIABILITY

Monitoring tools help organizations detect issues and measure whether SLA and SLO targets are being achieved.

Key Uses

- Detect and troubleshoot incidents
- Measure SLA compliance
- Track SLO performance
- Identify reliability trends



OBSERVABILITY PILLARS OVERVIEW



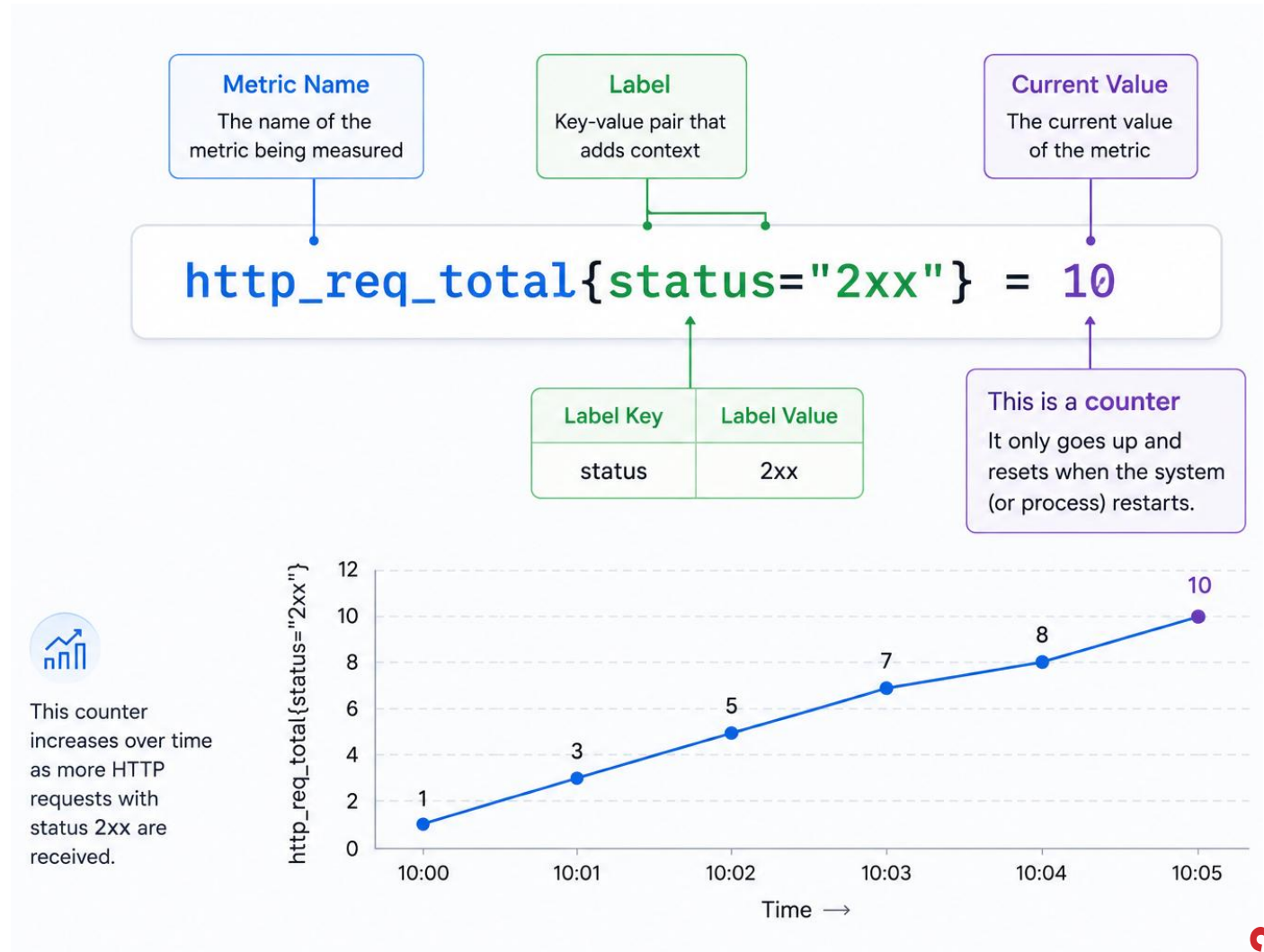
Observability is built on three core data types that explain system behavior. Together they provide context, depth, and confidence during incidents.

- Metrics
- Logs
- Traces
- Used together, not in isolation

METRICS

Metrics are numeric measurements collected over time. They are the fastest way to detect abnormal system behavior.

- CPU, memory, disk usage
- Request rates and error rates
- Latency and throughput
- Efficient and low cost
- Useful for alerts



Types of Metrics

Metrics describe different kinds of measurements in a system.



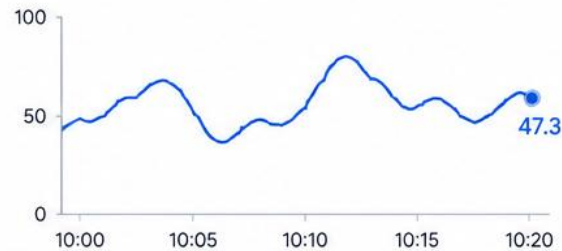
GAUGE

Represents a value that can go up and down.

Example

```
cpu_usage_percent = 47.3
```

Visualization



Key Characteristics

- Can increase or decrease
- Represents a point-in-time value
- Examples: CPU usage, memory usage, temperature, queue length



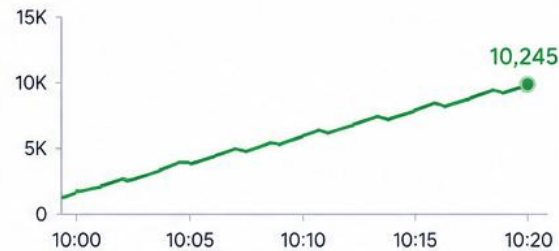
COUNTER

Represents a value that only increases (resets to 0 on restart).

Example

```
http_requests_total{status="2xx"} = 10245
```

Visualization



Key Characteristics

- Only increases over time
- Resets to 0 when the system restarts
- Examples: request count, error count, processed jobs



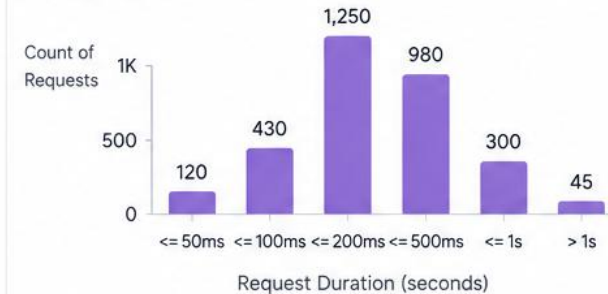
HISTOGRAM

Represents the distribution of observations over predefined buckets.

Example

```
http_request_duration_seconds_bucket
```

Visualization



Key Characteristics

- Captures the distribution of values
- Useful for understanding latency
- Examples: request duration, response size

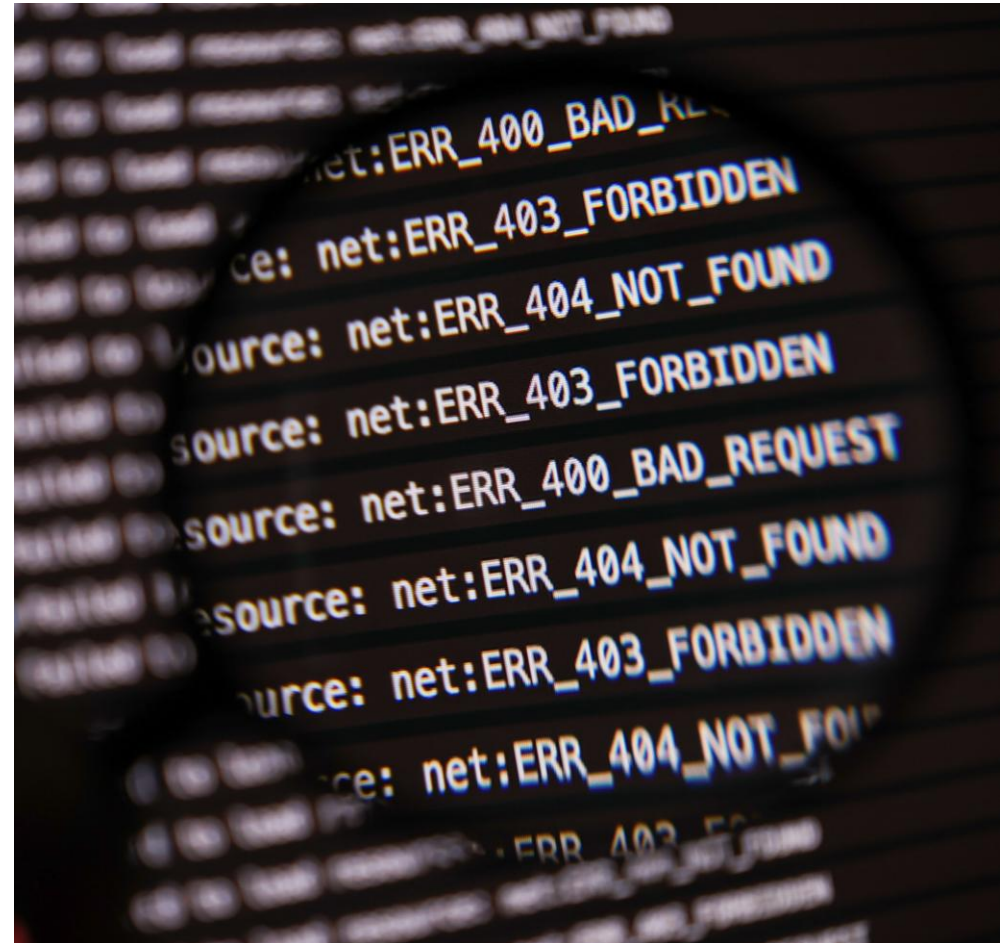


Why it matters: Different metric types help you capture different aspects of your system. Use the right type to get the most accurate and useful insights.

LOGS

Logs are timestamped records of events and messages. They explain what happened when something goes wrong.

- Application logs
- System and service logs
- Structured or unstructured text
- High detail, higher volume

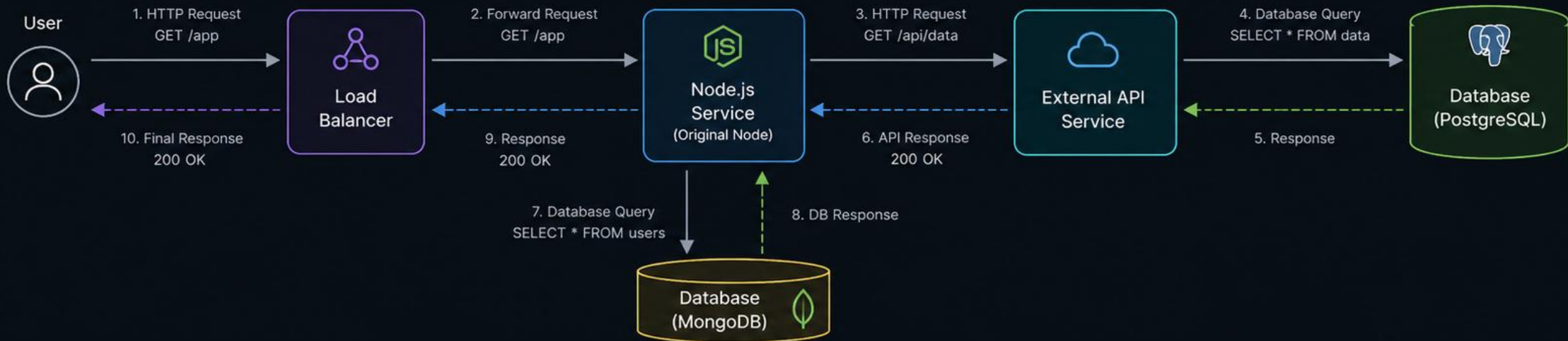


TRACES

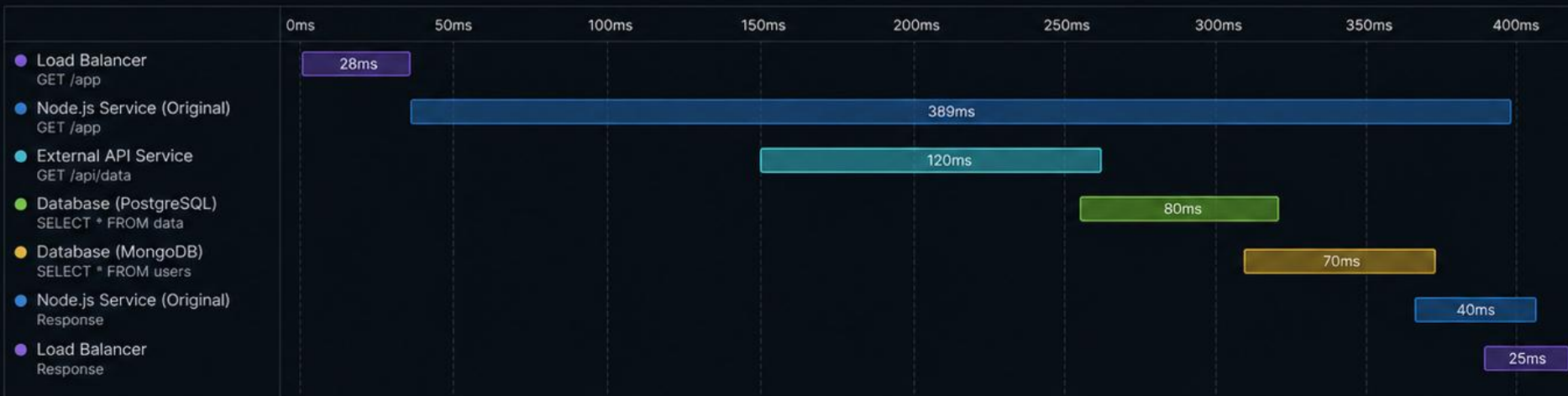
Traces show how a single request flows through a system. They are essential for debugging distributed architectures.

- End-to-end request visibility
- Service-to-service latency
- Dependency mapping
- Critical for microservices





Trace: 4f9c2a1e8b3d7c2b TRACE Total Duration: 412ms Start Time: 2024-05-21 12:34:56.789



Trace Details

Trace ID 4f9c2a1e8b3d7c2b
Total Duration 412ms
Span Count 7
Status OK

Tags

http.method GET
http.status_code 200
user.id 1234
env production

Resource

service.name node-service
service.version 1.2.3

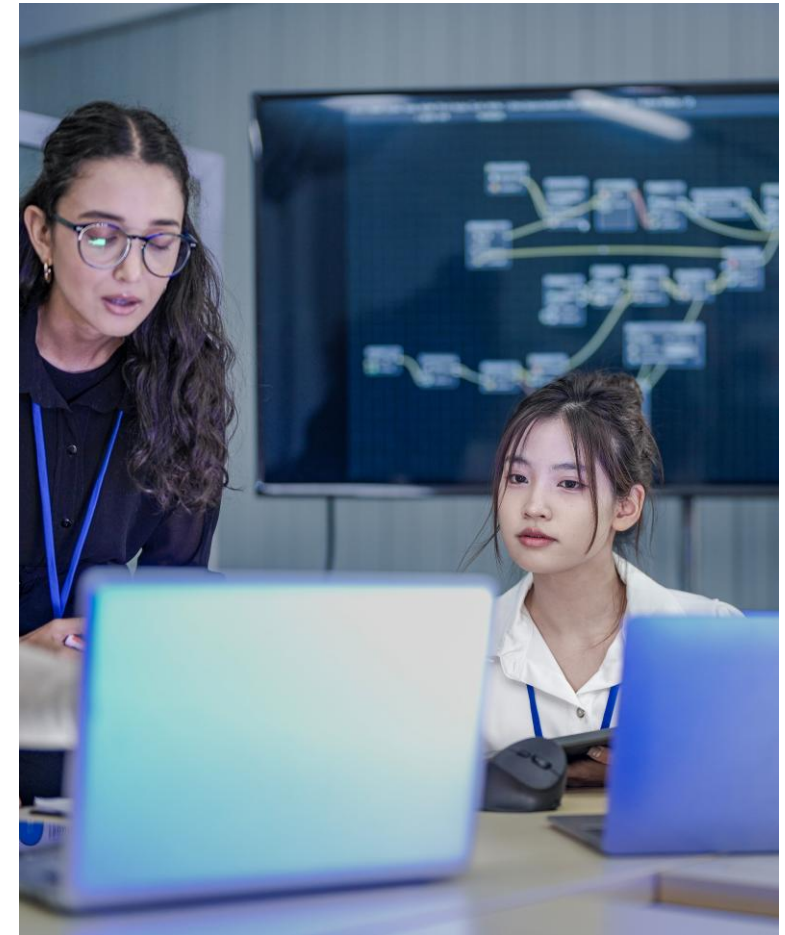


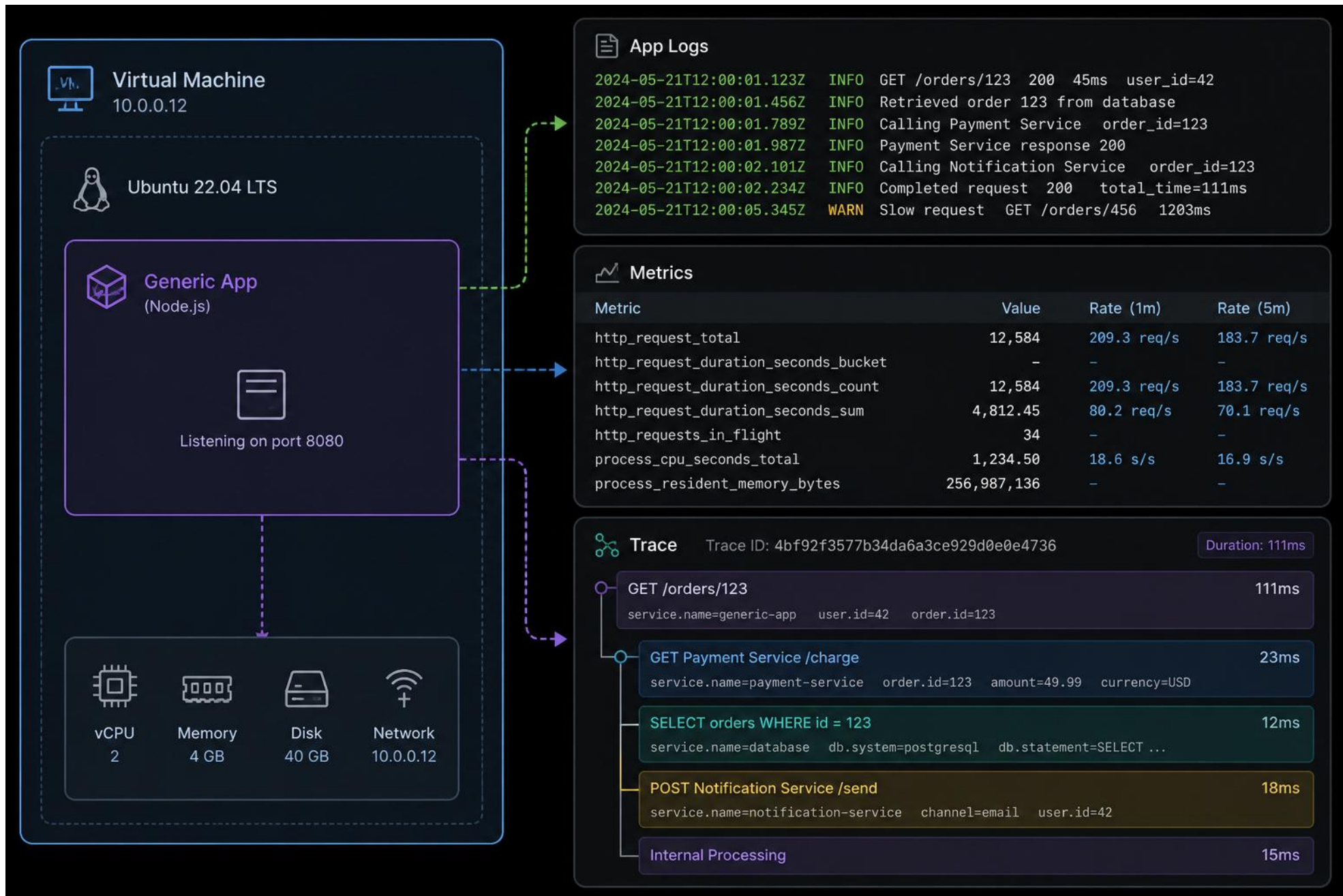
This is a single Trace representing the end-to-end request from the user through the system. Each colored bar is a Span representing a single operation.

WHY ALL THREE MATTER

Each pillar answers a different operational question. Using only one creates blind spots.

- Metrics show something is wrong
- Logs explain why
- Traces show where
- Correlation is key





POP QUIZ: Monitoring

Which observability pillar is best for detecting abnormal behavior quickly?

- A. Logs
- B. Metrics
- C. Traces
- D. Dashboards



POP QUIZ: Monitoring

Which observability pillar is best for detecting abnormal behavior quickly?

- A. Logs
- **B. Metrics**
- C. Traces
- D. Dashboards



POP QUIZ: Monitoring

What observability pillar helps explain why an error occurred?

- A. Metrics
- B. Logs
- C. Traces
- D. Alerts



POP QUIZ: Monitoring

What observability pillar helps explain why an error occurred?

- A. Metrics
- **B. Logs**
- C. Traces
- D. Alerts



POP QUIZ: Monitoring

What data type shows how a request flows across multiple services?

- A. Metrics
- B. Logs
- C. Traces
- D. Dashboards



POP QUIZ: Monitoring

What data type shows how a request flows across multiple services?

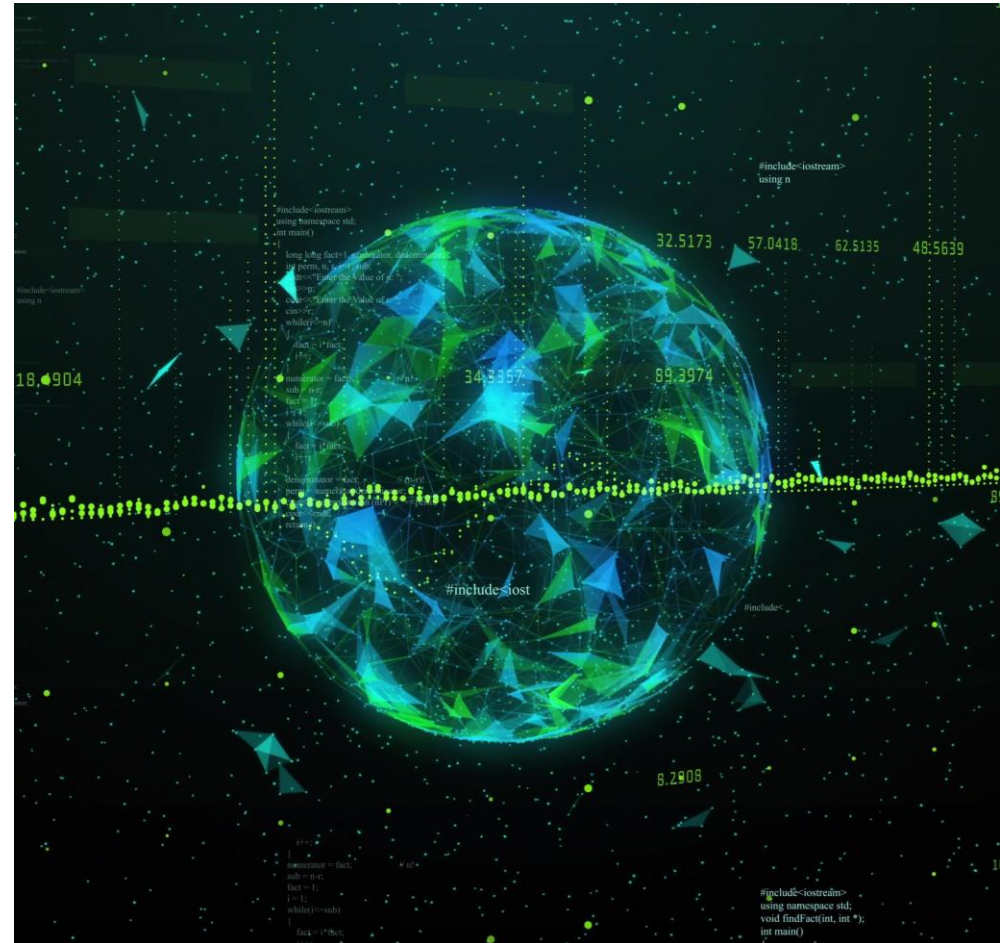
- A. Metrics
- B. Logs
- **C. Traces**
- D. Dashboards



TELEMETRY FUNDAMENTALS

Telemetry is the concept of an application emitting data about its own behavior. This data can later be collected and analyzed by external systems.

- Emitted by the application itself
- Collected by monitoring tools
- Independent of vendors
- Core to observability

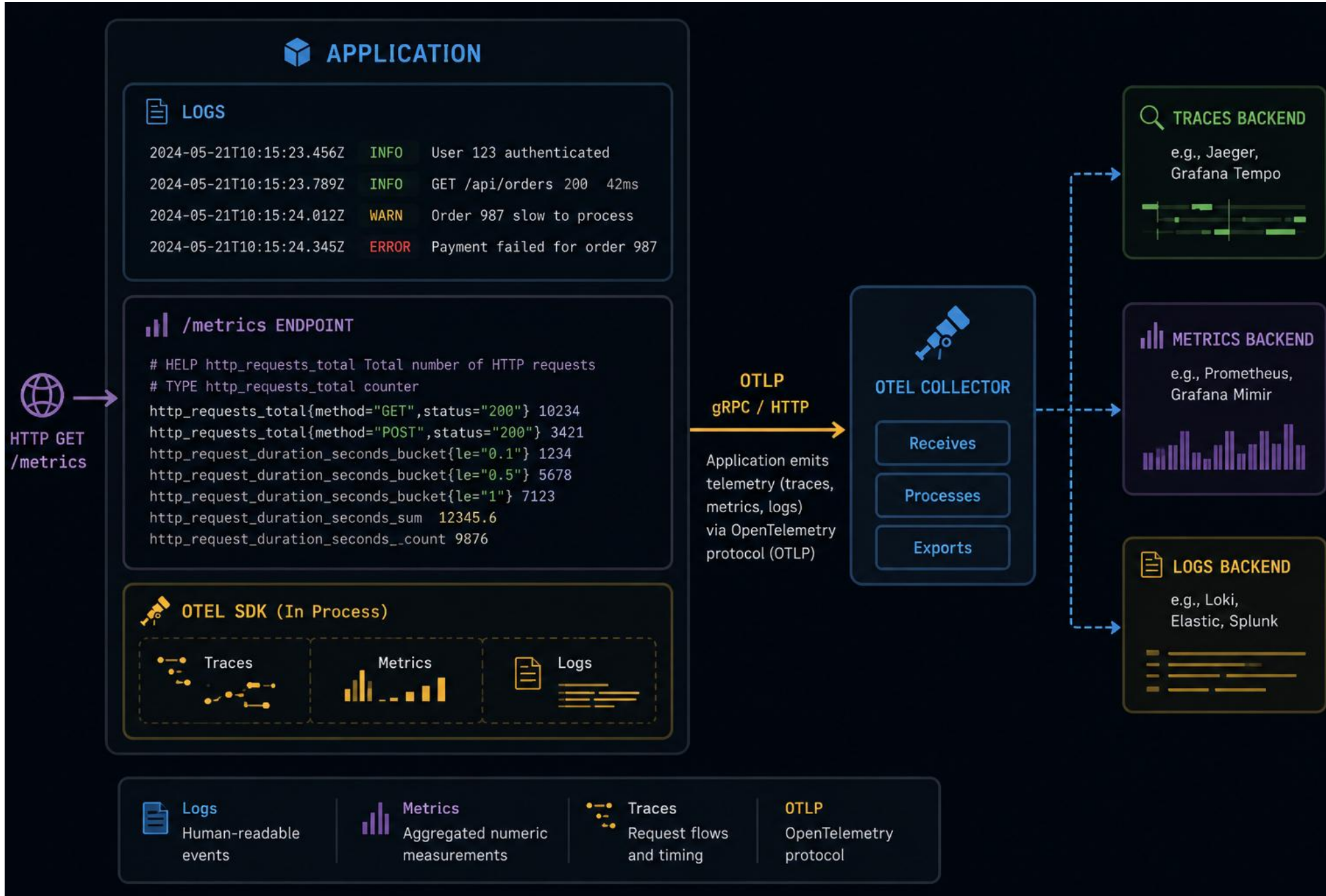


TELEMETRY VS MONITORING TOOLS

Telemetry is not a tool, database, or dashboard. It is the raw signal that monitoring systems consume.

- Application responsibility
- Tool agnostic
- Exists before dashboards
- Enables observability





OPENTELEMETRY (OTEL)

OpenTelemetry is an open standard for collecting and sending telemetry data from applications and infrastructure. It provides a vendor-neutral way to generate logs, metrics, and traces that can be sent to observability platforms.

- Open standard and CNCF project
- Collects logs, metrics, and traces
- Vendor-neutral telemetry format
- Integrates with tools such as Grafana, Dynatrace, and Splunk

Key Benefit: Instrument once, send data anywhere.

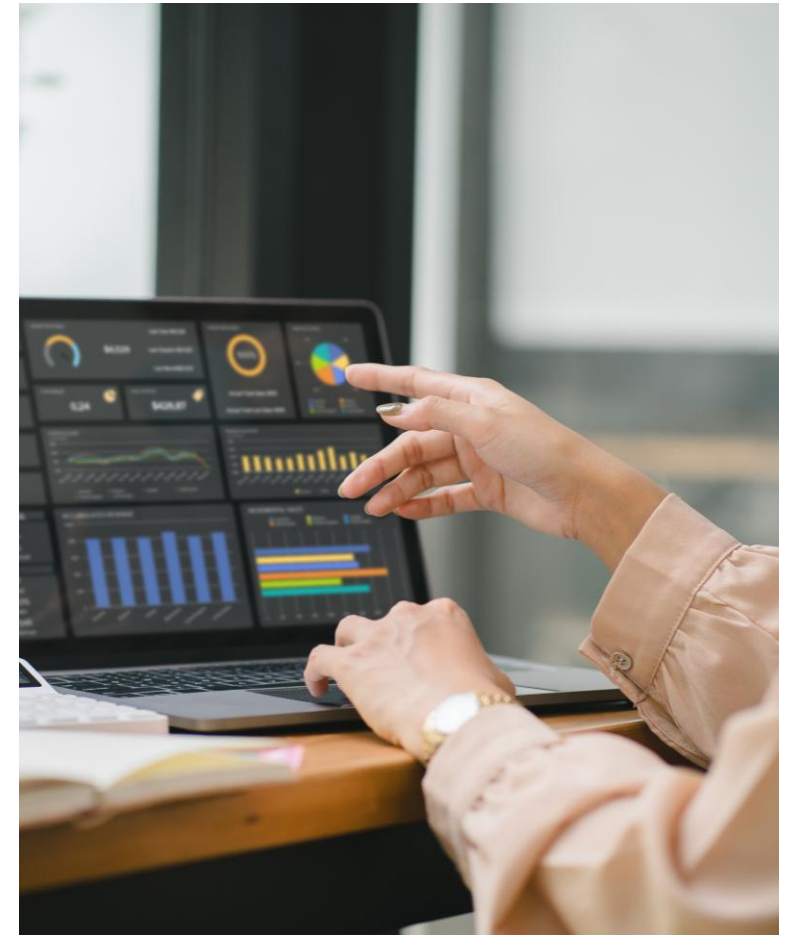


TRADITIONAL METRICS VS OPENTELEMETRY

Traditional monitoring tools such as Prometheus use a **pull model**, where the monitoring platform periodically scrapes a /metrics endpoint exposed by the application.

OpenTelemetry uses a **push model**, where telemetry data is generated inside the application and sent directly to a collector or observability platform.

It is common to use /metrics (not OTEL) for metrics data, and to use OTEL for traces. Although, you can use OTEL for metrics, traces and logs.



TRADITIONAL METRICS VS OPENTELEMETRY

Prometheus pulls metrics from applications, while OTEL instrumentation pushes telemetry data from within the application to a collector or backend.

Traditional Metrics (Pull)	OpenTelemetry (Push)
Exposes /metrics endpoint	Application exports telemetry
Monitoring system scrapes data	Application sends data
Common for Prometheus metrics	Common for traces and OTLP
Monitoring initiates collection	Application initiates collection

ALERTING

Alerting notifies engineers when service health or reliability targets are at risk. Alerts are commonly created from SLA/SLO requirements, application metrics, and infrastructure health indicators.

- High error rates
- Increased latency
- CPU or memory exhaustion
- Service availability failures

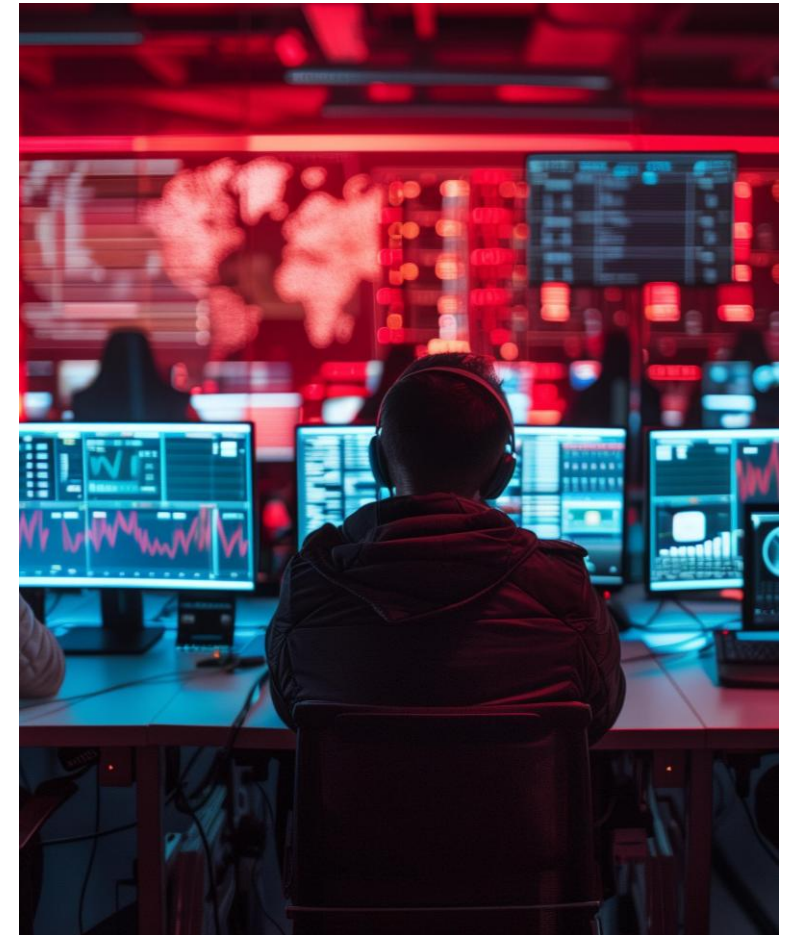
Takeaway: Monitoring tools not only generate alerts, but also provide the metrics, logs, and traces needed to diagnose and resolve incidents quickly.



MTTD AND MTTR

Good monitoring reduces how long issues go unnoticed. Observability reduces how long it takes to fix them.

- MTTD: Mean Time To Detect
- MTTR: Mean Time To Resolve
- Faster detection saves downtime
- Context speeds resolution
- Good alerts reduce MTTD



DYNATRACE

Dynatrace is an observability platform that provides unified monitoring across applications, infrastructure, logs, traces, and user experiences.

- Automatic service discovery
- AI-assisted root cause analysis
- Metrics, logs, traces, and alerts
- End-to-end application visibility



POP QUIZ: Monitoring

Why is monitoring required to track SLOs and SLAs?

- A. It stores application data
- B. It provides measurable reliability data
- C. It replaces incident response
- D. It eliminates downtime



POP QUIZ: Monitoring

Why is monitoring required to track SLOs and SLAs?

- A. It stores application data
- **B. It provides measurable reliability data**
- C. It replaces incident response
- D. It eliminates downtime



POP QUIZ: Monitoring

What best describes telemetry?

- A. A monitoring dashboard
- B. An application emitting data about itself
- C. A logging format
- D. A metrics database



POP QUIZ: Monitoring

What best describes telemetry?

- A. A monitoring dashboard
- **B. An application emitting data about itself**
- C. A logging format
- D. A metrics database



POP QUIZ: Monitoring

Which telemetry type is primarily used to calculate SLOs?

- A. Logs
- B. Traces
- C. Metrics
- D. Alerts



POP QUIZ: Monitoring

Which telemetry type is primarily used to calculate SLOs?

- A. Logs
- B. Traces
- **C. Metrics**
- D. Alerts



POP QUIZ: Monitoring

Why does telemetry reduce MTTR?

- A. It replaces incident response
- B. It provides context and visibility
- C. It prevents failures entirely
- D. It scales infrastructure automatically



POP QUIZ: Monitoring

Why does telemetry reduce MTTR?

- A. It replaces incident response
- **B. It provides context and visibility**
- C. It prevents failures entirely
- D. It scales infrastructure automatically



POP QUIZ: Monitoring

How do monitoring tools reduce MTTR?

- A. It replaces incident response
- B. It provides context and visibility
- C. It prevents failures entirely
- D. It scales infrastructure automatically



POP QUIZ: Monitoring

How do monitoring tools reduce MTTD?

- A. Logs help detect problems quicker
- B. Monitoring tools with AI help resolve incidents
- C. It scales infrastructure automatically
- D. Real-time alerts notify staff when problems occur



POP QUIZ: Monitoring

How do monitoring tools reduce MTTR?

- A. Logs help detect problems quicker
- B. Monitoring tools with AI help resolve incidents
- C. It scales infrastructure automatically
- D. Real-time alerts notify staff when problems occur**



POP QUIZ: Monitoring

What is the purpose of the /metrics endpoint?

- A. To expose logs
- B. To store metrics
- C. To expose telemetry for scraping
- D. To visualize dashboards



POP QUIZ: Monitoring

What is the purpose of the /metrics endpoint?

- A. To expose logs
- B. To store metrics
- **C. To expose telemetry for scraping**
- D. To visualize dashboards



LAB 3.1



LOGSCALE

Logscale is a centralized log management platform used to collect, search, and analyze large volumes of log data. Logscale is CrowdStrike's commercial log platform

- Centralized log storage
- Fast search and filtering
- Real-time log analysis
- Supports operational and security use cases

Takeaway: Logscale helps teams turn raw log data into actionable insights.



INTRODUCTION TO GRAFANA

Grafana is an **open-source platform** used to visualize, monitor, and explore data from many sources. It is a core tool for modern observability and incident response workflows.

- Web-based visualization platform
- Open source and widely adopted
- Used by SRE and platform teams
- Focused on observability use cases
- Paid enterprise plans are available



WHAT GRAFANA IS

Grafana turns raw data into dashboards made of graphs, charts, and alerts. It helps teams understand system behavior at a glance.

- Dashboards and panels
- Time-series focused
- Query-based visualizations
- Supports alerting



WHY GRAFANA IS USED

Grafana is commonly used to monitor applications and infrastructure in real time. It helps teams detect issues early and understand system behavior.

- Application monitoring
- Infrastructure visibility
- Real-time dashboards
- Shared team views



WHAT GRAFANA IS NOT

Grafana does not store metrics or logs by itself in most cases. It relies on external data sources to provide information.

- Not a database
- Not a metrics collector
- Not a log storage engine
- Acts as a visualization layer



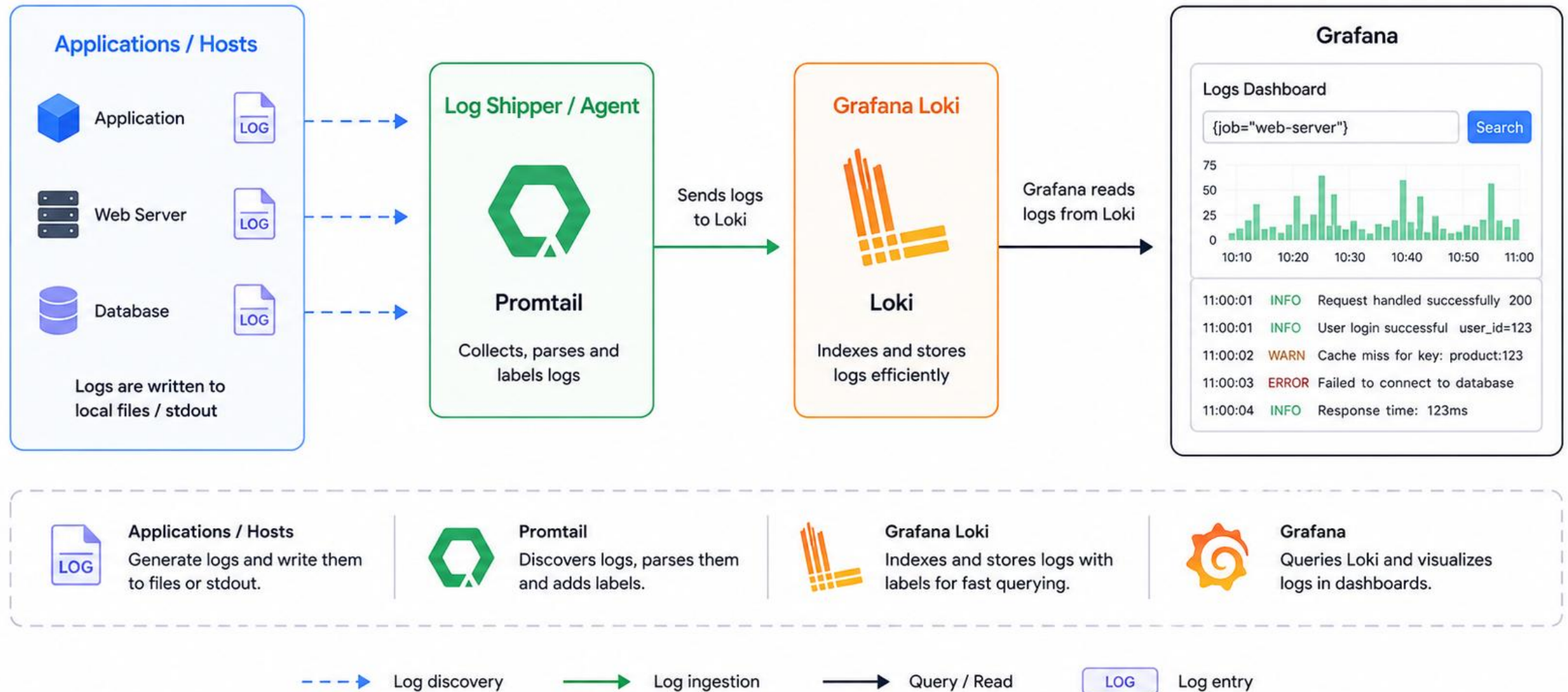
LGTM STACK (LOKI)

The LGTM (Loki, Grafana, Tempo, Mimir) stack provides a 360 view of applications

Loki is a Time Series Log database. LogQL is used to write queries against the logs. Logs are useful for RCA (root cause analysis), troubleshooting and detecting issues.



Logs with Loki



LGTM STACK (PROMETHEUS)

Prometheus stores time-series metric data for monitoring systems. Mimir extends Prometheus into a scalable, distributed backend for long-term storage.

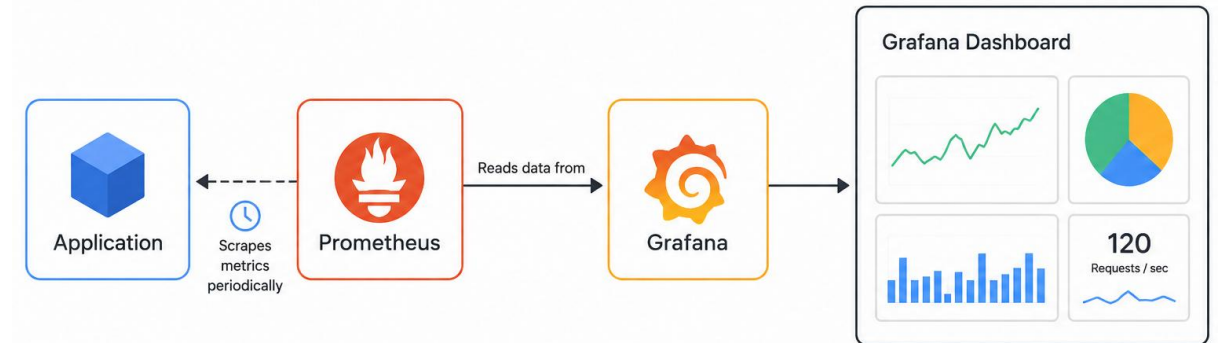
Metrics, being structured and numeric, enable reliable tracking of SLOs, latency, and application performance.



PROMETHEUS

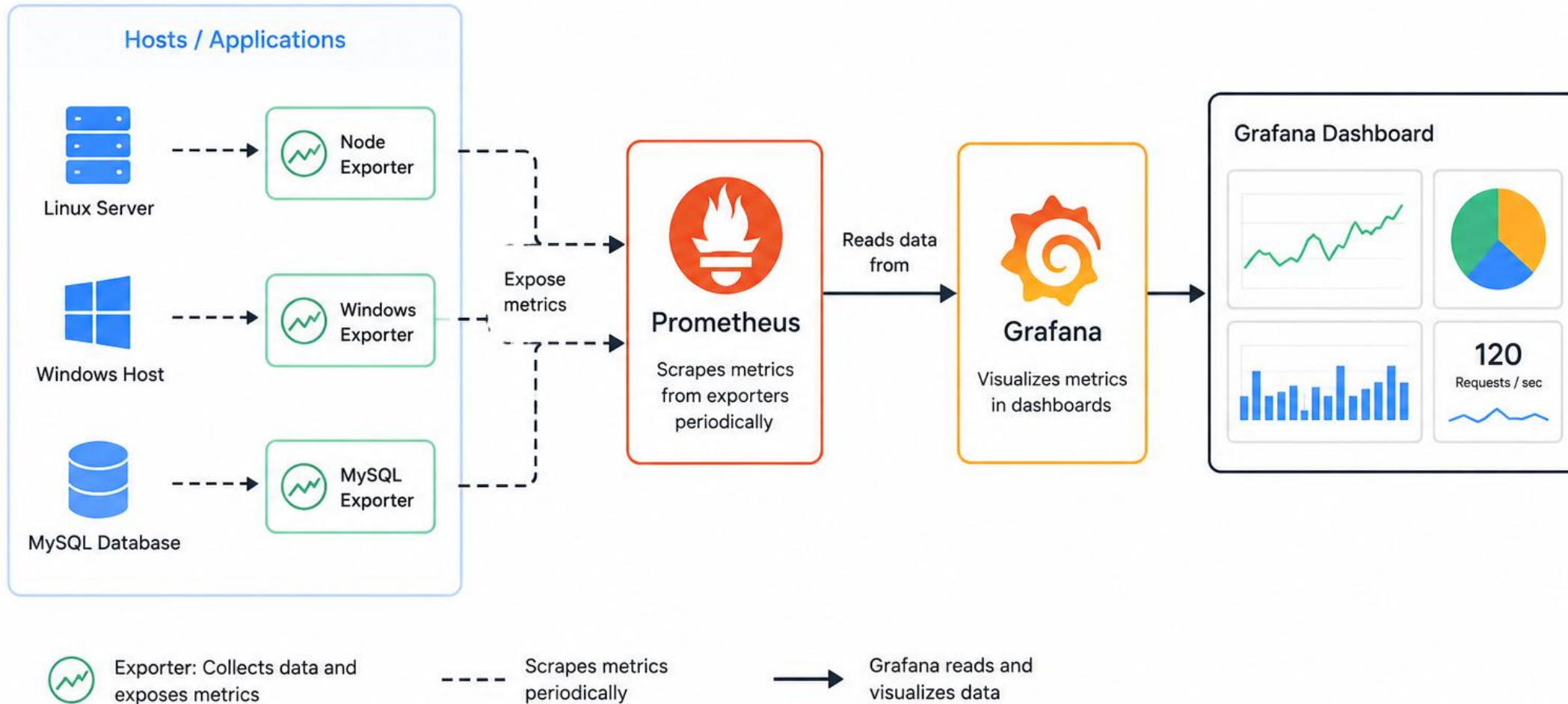
Prometheus collects metrics from applications by periodically scraping their /metrics endpoints.

Grafana reads the stored metrics from Prometheus and visualizes them through dashboards, helping teams monitor system health and performance trends.



Application Exporters

Some applications and systems don't expose built-in metrics in a format Prometheus can scrape. Exporters run on the host to collect data and expose it as metrics for Prometheus.



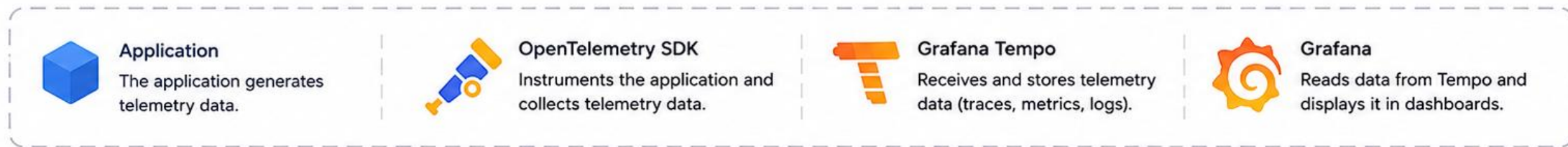
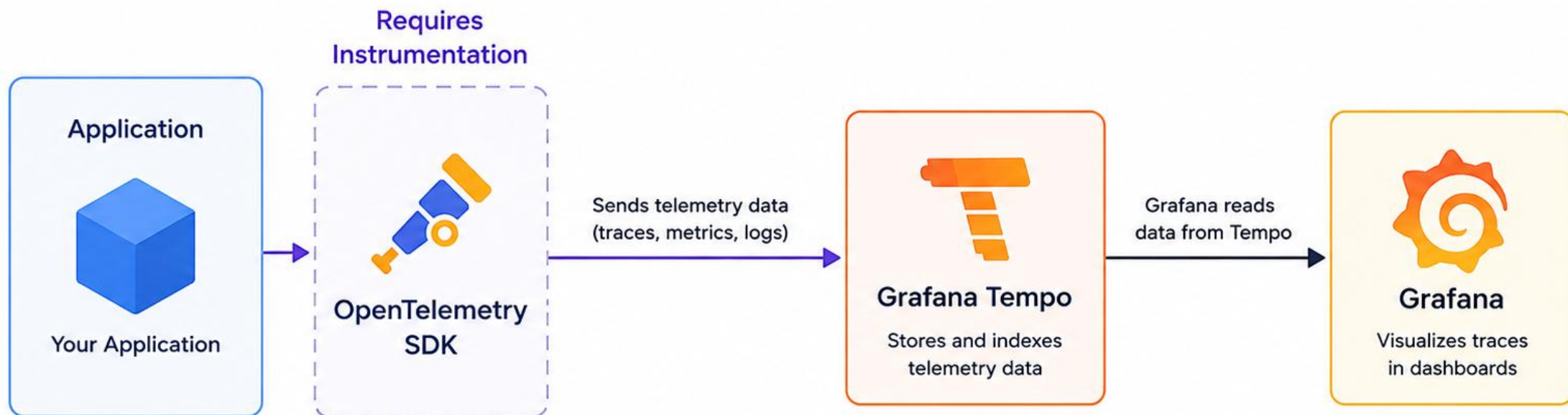
LGTM STACK (TEMPO)

Tempo stores distributed traces at scale. Traces provide end-to-end visibility into request flow, dependencies, and latency across services.

Metrics may give you the result of a whole requests, but traces break it into pieces called Spans.



OTEL TRACES AND TEMPO

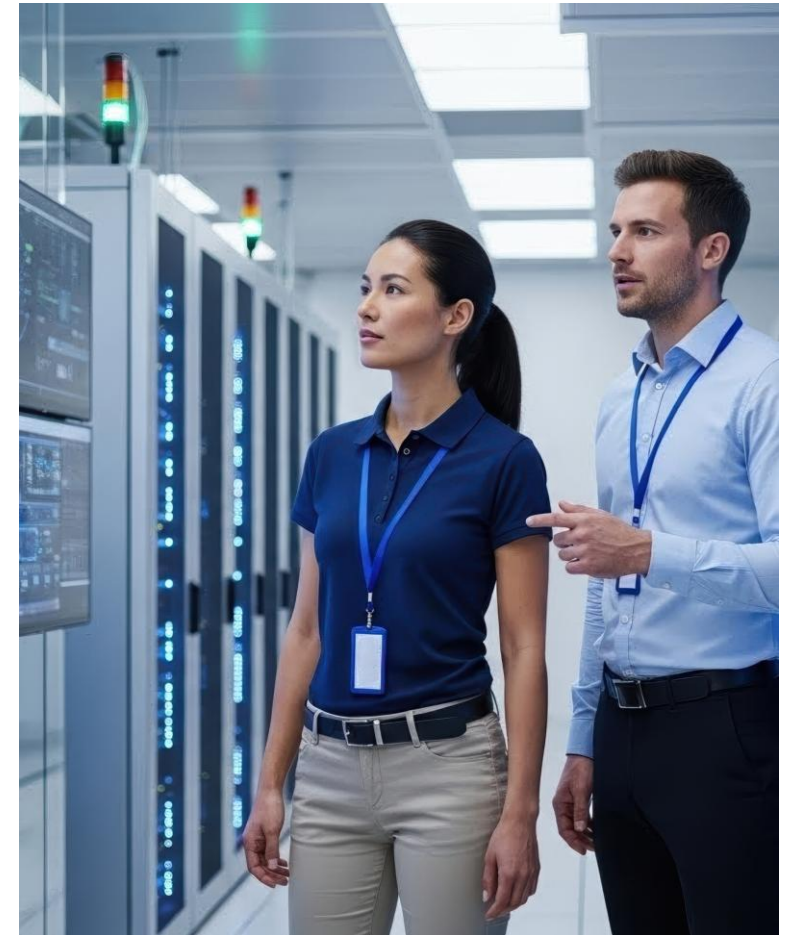


→ Instrumentation → Telemetry data flow → Query / Read

WHY CENTRALIZED LOGGING

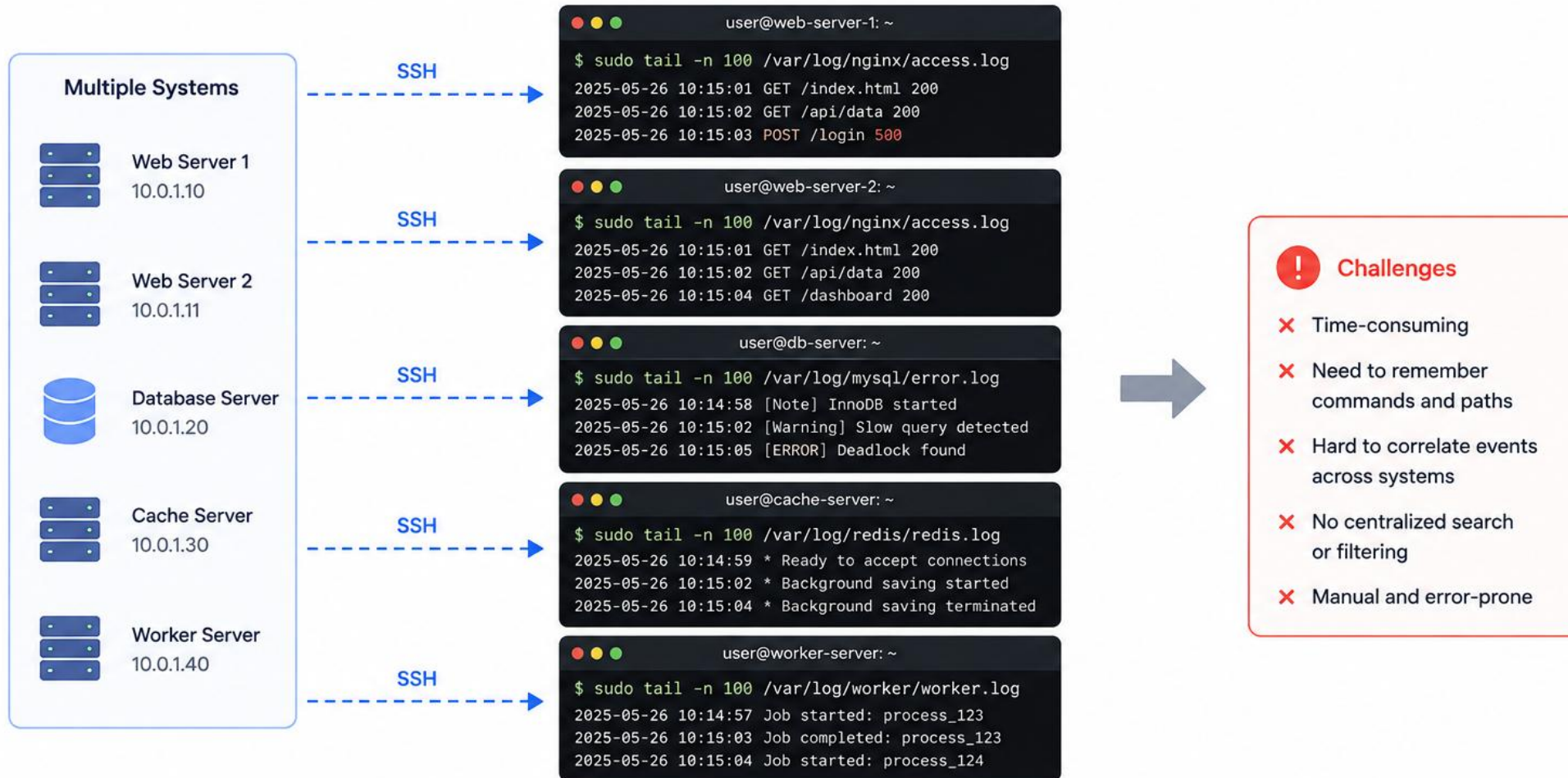
Logs are most valuable when they are searchable and correlated. Centralized logging removes the need to access individual servers.

- Single source of truth
- Searchable history
- Cross-service visibility
- Faster debugging



Gathering Logs the Hard Way

SSH into multiple systems and run CLI commands to find logs

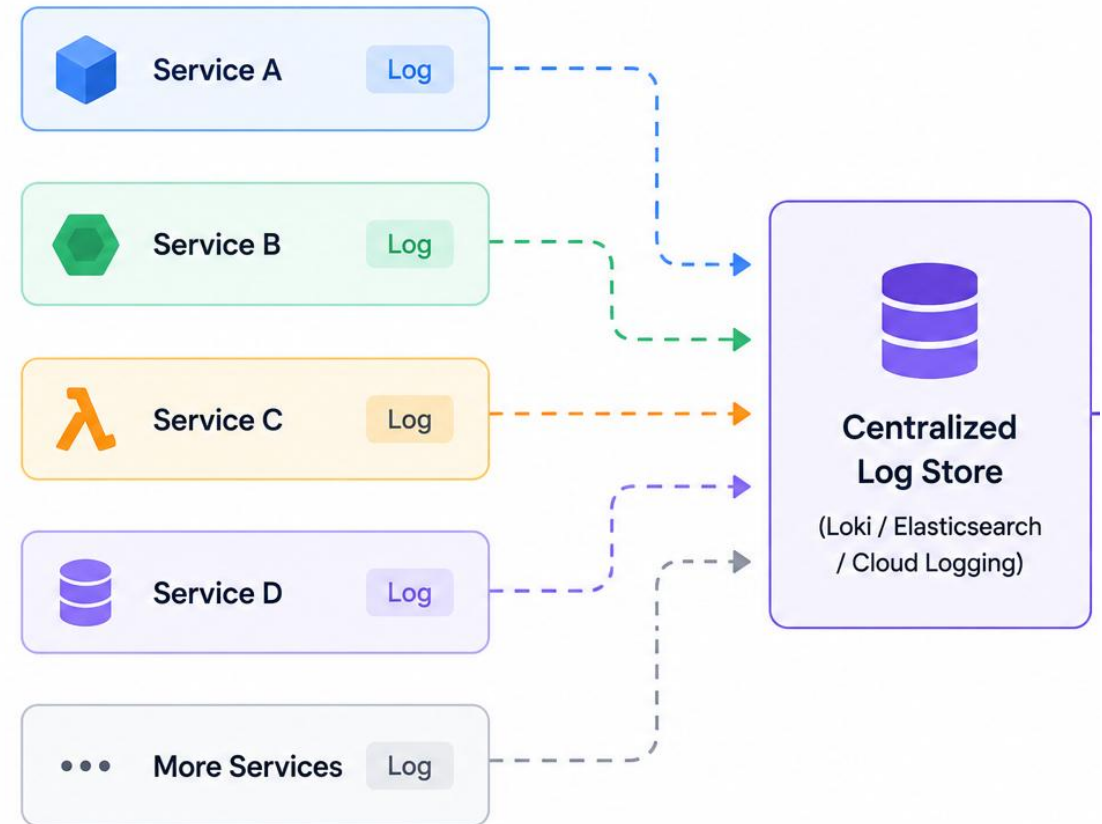


Without centralized logging, finding answers means jumping between servers and commands. It's slow, inconsistent, and hard to scale.

CENTRALIZED LOGGING OVERVIEW

Centralized logging collects logs from many services into one place. This makes searching, correlating, and retaining logs practical at scale.

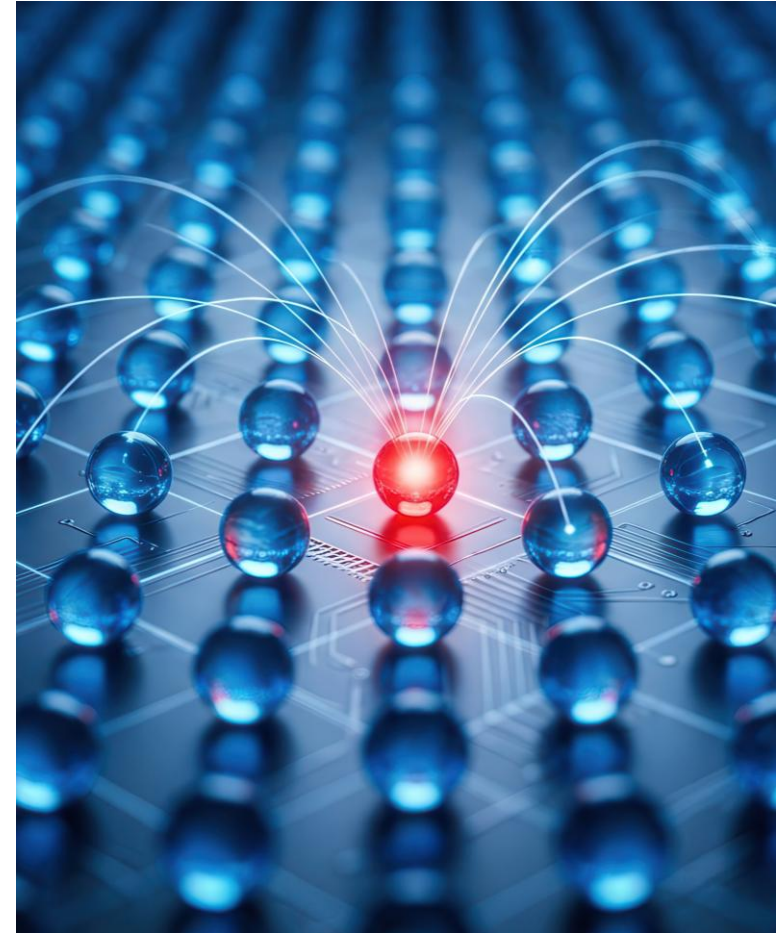
- Single log location
- Searchable history
- Multi-service visibility
- Operational at scale



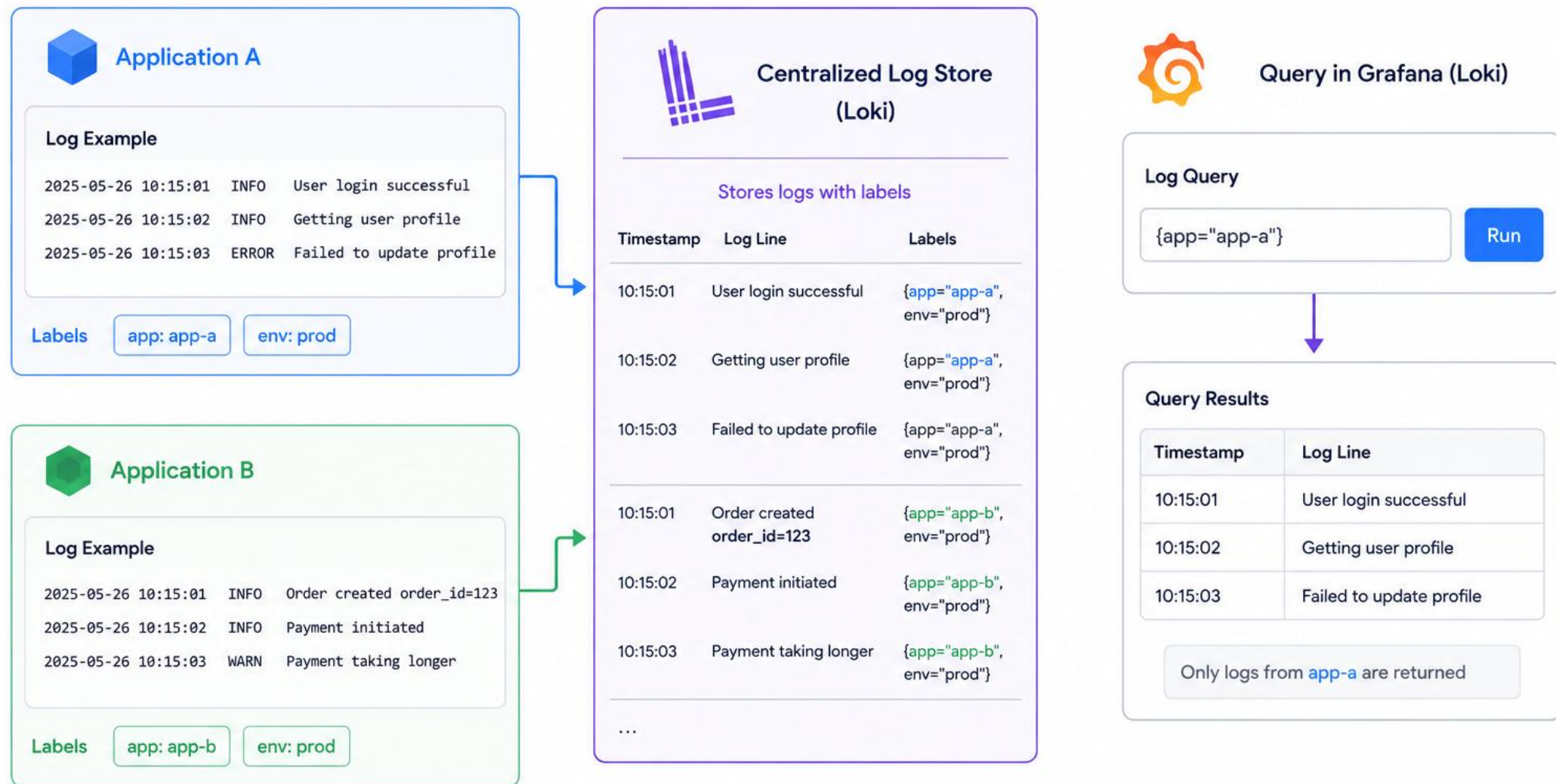
LOGGING AT SCALE PROBLEM

Modern systems run many containers across many hosts. Logs must be centralized to remain useful.

- Many containers
- Frequent restarts
- Distributed systems
- Shared responsibility



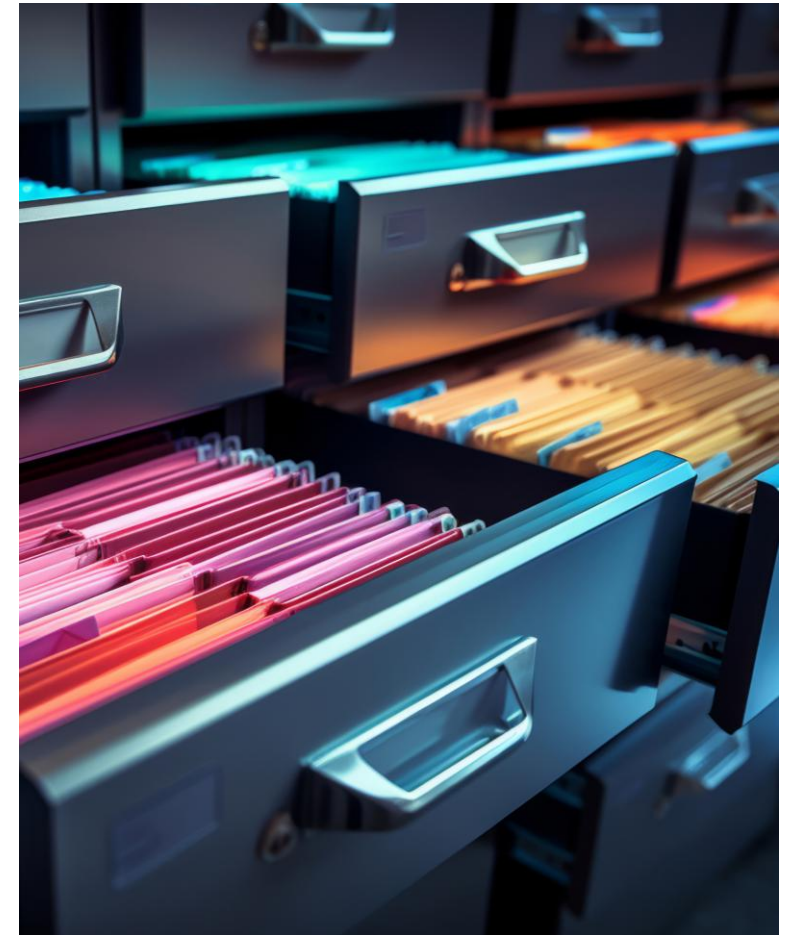
LABEL BASED INDEXING



LOKI DESIGN PHILOSOPHY

Loki (part of Grafana) indexes metadata instead of full log contents. This makes it simpler and cheaper than traditional logging systems.

- Labels not full text
- Low storage cost
- Fast queries
- Observability focused



GRAFANA AND LOGS

Grafana provides a UI to explore and query logs stored in Loki. This removes the need for SSH access to servers.

- Explore view
- Search and filter
- Time based queries
- Team accessible



LOGS AS TELEMETRY SIGNALS

Logs are a form of telemetry emitted by applications. They describe events that occurred during execution.

- Application emitted
- Event driven
- Human readable
- Operational signal



LOGS DURING INCIDENTS

Logs provide context when metrics indicate a problem. They help explain what happened and why.

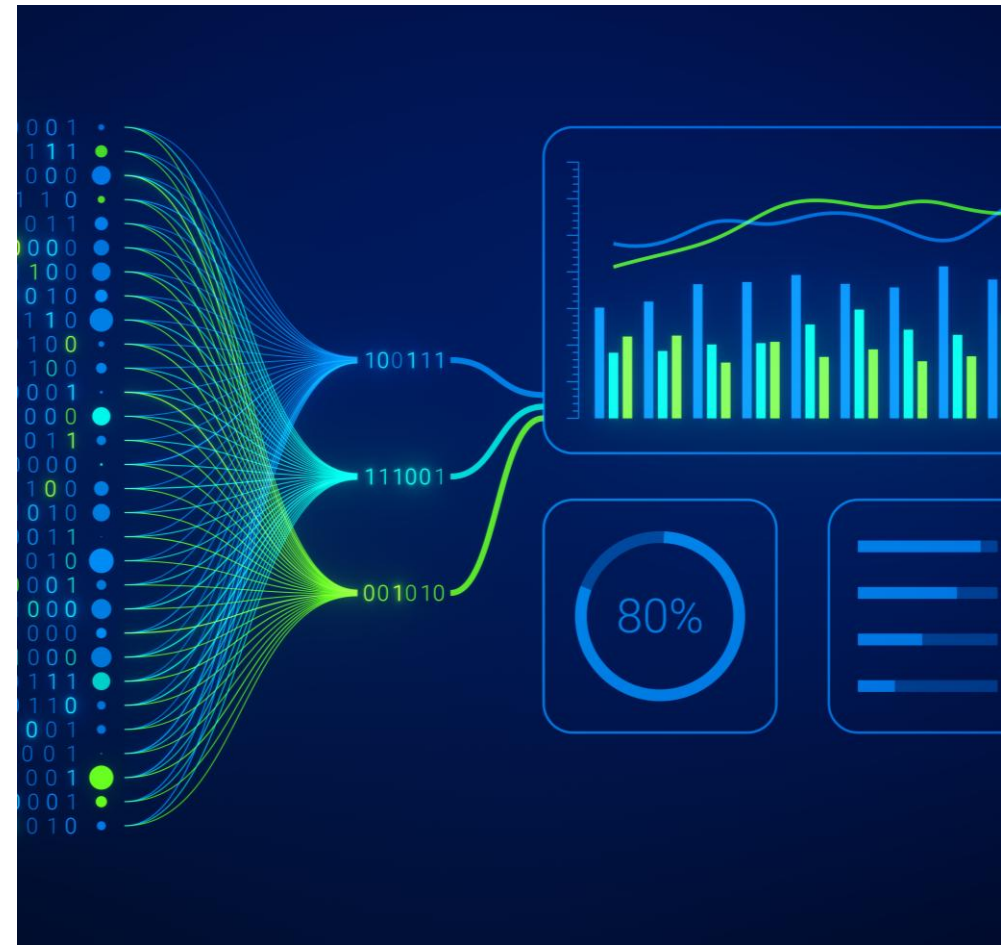
- Error messages
- Warnings
- Execution flow
- Root cause clues



LOGS AND METRICS TOGETHER

Metrics tell you that something is wrong. Logs help explain why it happened.

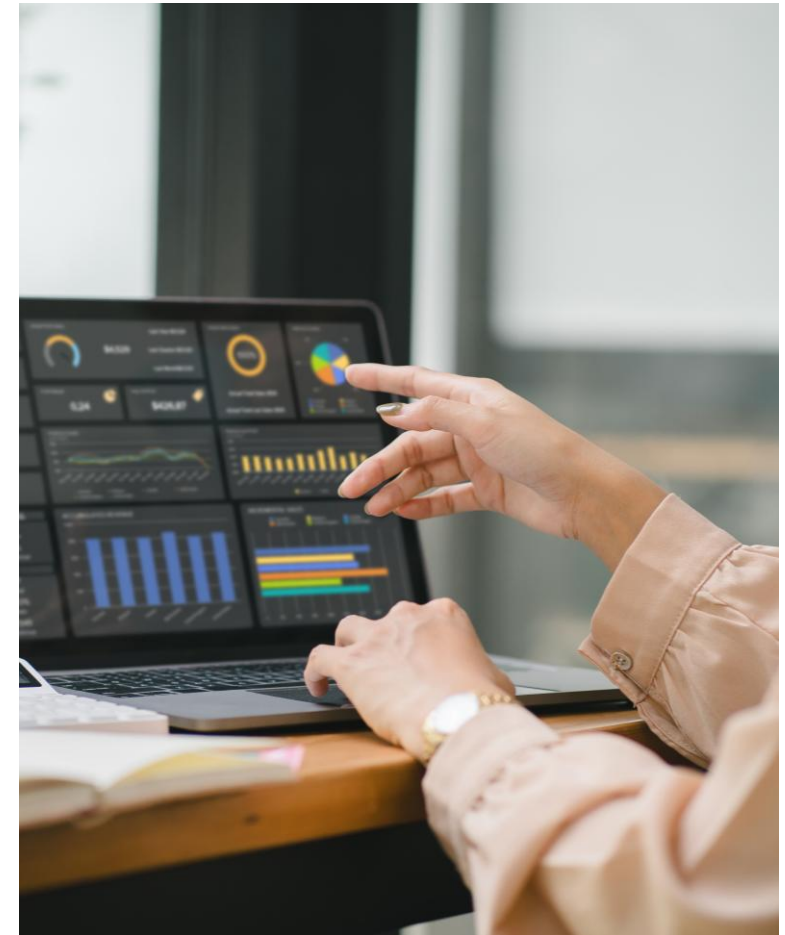
- Metrics detect
- Logs explain
- Complementary signals
- Correlated analysis



LOGS IN POST INCIDENT REVIEWS

Logs provide factual timelines after incidents. They support learning and improvement.

- Event timelines
- Root cause analysis
- Evidence based reviews
- Reliability improvement



POP QUIZ: Centralized Logging

What problem does centralized logging primarily solve?

- A. Reducing application latency
- B. Searching and correlating logs across services
- C. Replacing metrics systems
- D. Eliminating application errors



POP QUIZ: Centralized Logging

What problem does centralized logging primarily solve?

- A. Reducing application latency
- **B. Searching and correlating logs across services**
- C. Replacing metrics systems
- D. Eliminating application errors



POP QUIZ: Centralized Logging

What is Loki optimized to index?

- A. Full log message text
- B. Application binaries
- C. Log metadata and labels
- D. Database records



POP QUIZ: Centralized Logging

What is Loki optimized to index?

- A. Full log message text
- B. Application binaries
- **C. Log metadata and labels**
- D. Database records



POP QUIZ: Centralized Logging

What is the main role of Promtail?

- A. Visualize logs
- B. Store logs long term
- C. Collect and ship logs to Loki
- D. Generate application logs



POP QUIZ: Centralized Logging

What is the main role of Promtail?

- A. Visualize logs
- B. Store logs long term
- **C. Collect and ship logs to Loki**
- D. Generate application logs



POP QUIZ: Centralized Logging

What is the first step in a LogQL query?

- A. Filter log lines
- B. Select log streams using labels
- C. Apply regex matching
- D. Aggregate results



POP QUIZ: Centralized Logging

What is the first step in a LogQL query?

- A. Filter log lines
- **B. Select log streams using labels**
- C. Apply regex matching
- D. Aggregate results



POP QUIZ: Centralized Logging

Why are labels important in Loki?

- A. They format log messages
- B. They allow efficient pre-filtering
- C. They replace log content
- D. They store metrics



POP QUIZ: Centralized Logging

Why are labels important in Loki?

- A. They format log messages
- **B. They allow efficient pre-filtering**
- C. They replace log content
- D. They store metrics



POP QUIZ: Centralized Logging

Which LGTM component is responsible for storing logs?

- A. Prometheus
- B. Tempo
- C. Loki
- D. Alloy



POP QUIZ: Centralized Logging

Which LGTM component is responsible for storing logs?

- A. Prometheus
- B. Tempo
- **C. Loki**
- D. Alloy



LAB 3.2



CONFIGURATION MANAGEMENT

Configuration management tools help maintain consistent system settings across environments.

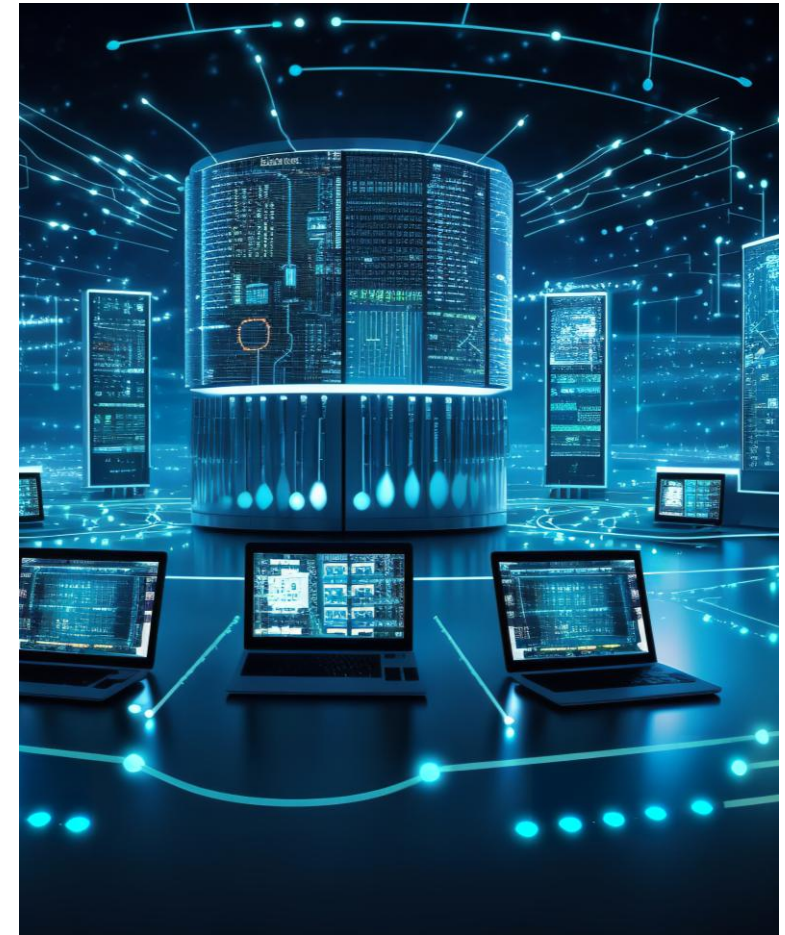
- Ansible applies desired configurations
- Reduces manual administration
- Promotes repeatability and standardization



INFRASTRUCTURE AS CODE (IAC)

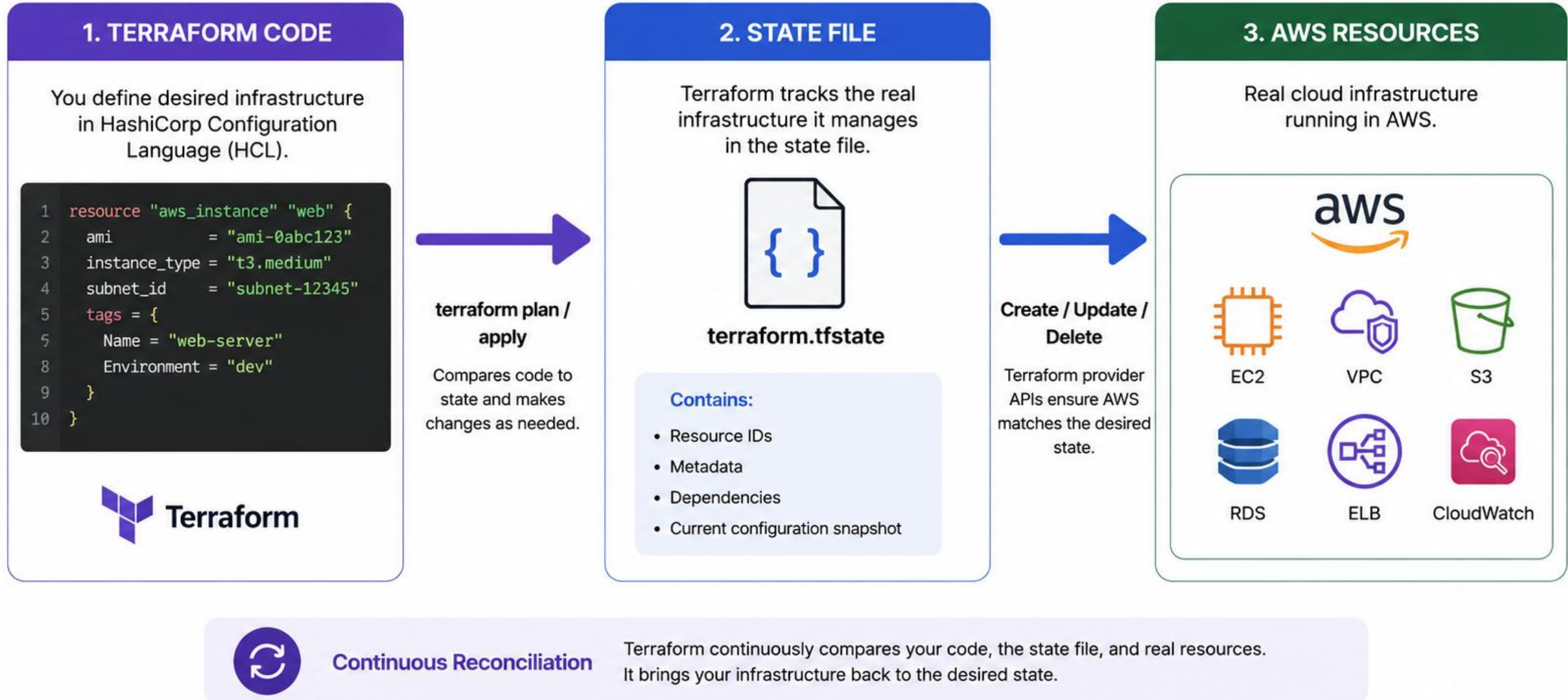
Infrastructure is defined in code rather than manually created through a console.

- Terraform defines cloud resources
- Version controlled and auditable
- Repeatable deployments



How Terraform Works

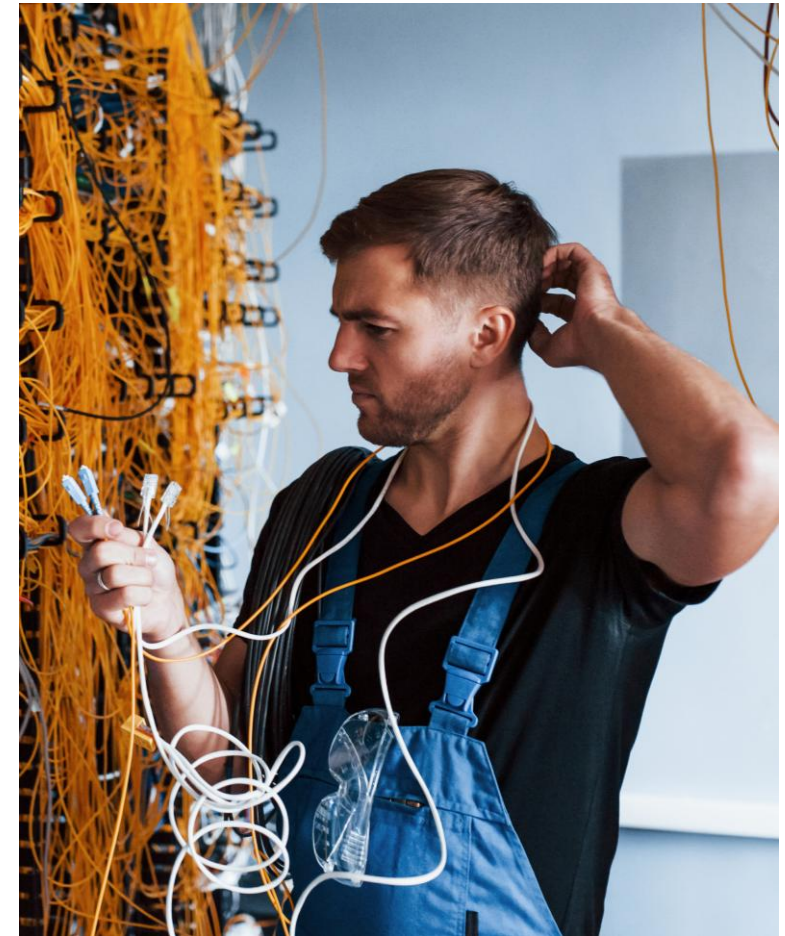
Terraform uses your code to define infrastructure, tracks what it created in the state file, and ensures your real resources match your configuration.



WHAT IS CONFIGURATION DRIFT?

Drift occurs when a live environment no longer matches the desired configuration.

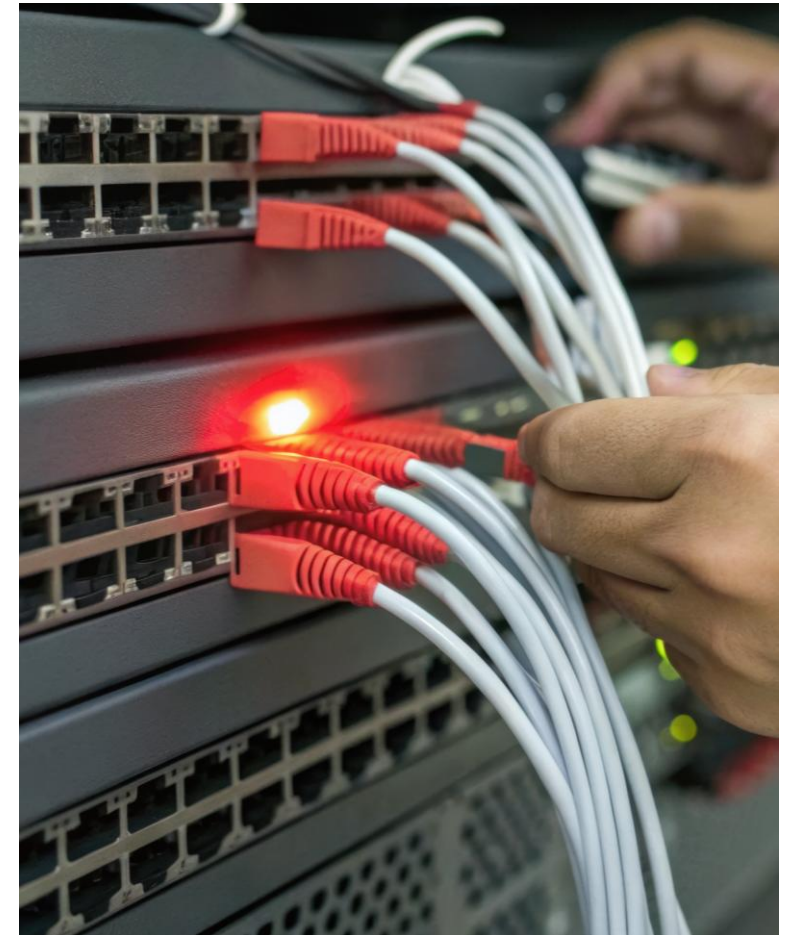
- Manual console changes
- Emergency production fixes
- Untracked modifications



EXAMPLES OF DRIFT

Common examples of drift in cloud environments.

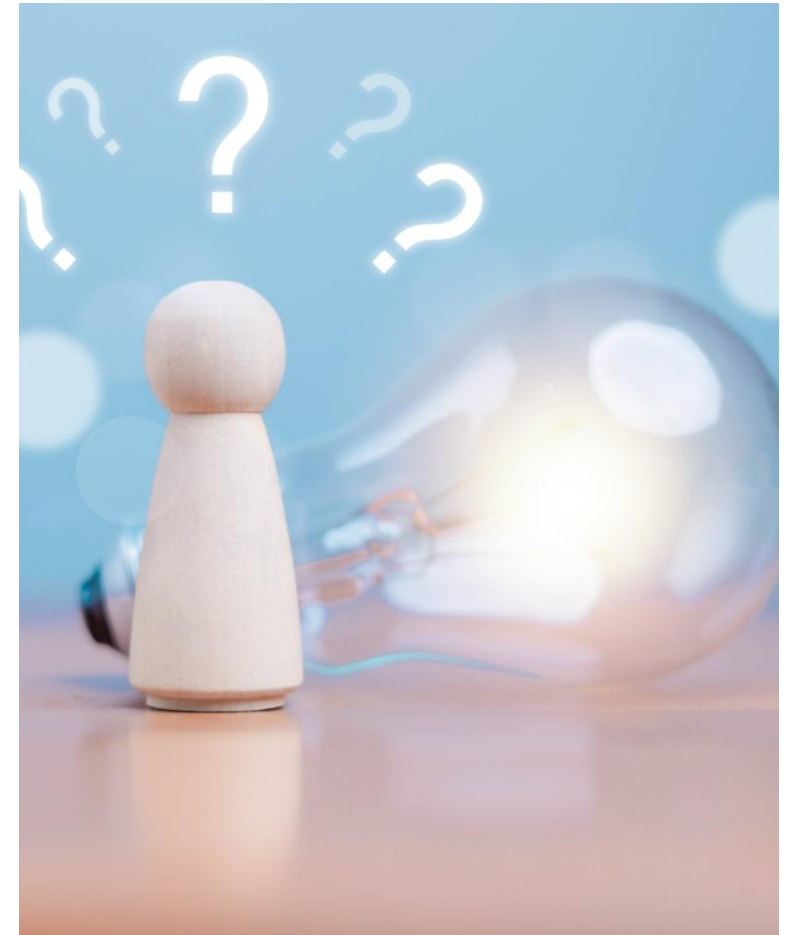
- Security groups modified manually
- EC2 instance sizes changed
- IAM permissions updated outside Terraform



WHY DRIFT HAPPENS

Drift happens when an update is required, but instead of doing so through the proper IaC or CM tool, you do it manually.

It is faster to simply update a server than to update the IaC and run the whole pipeline again. Especially when testing different changes.



DETECTING DRIFT WITH HCP TERRAFORM

Platforms such as HCP Terraform continuously compare deployed resources against code.

- Automated drift detection
- Visibility into unauthorized changes
- Governance and compliance benefits



TERRAFORM STATE FILE

Terraform uses a state file to track deployed infrastructure.

- Maps resources to cloud objects
- Enables change detection
- Required for drift analysis



REMOTE BACKENDS AND S3

Teams commonly store Terraform state remotely.

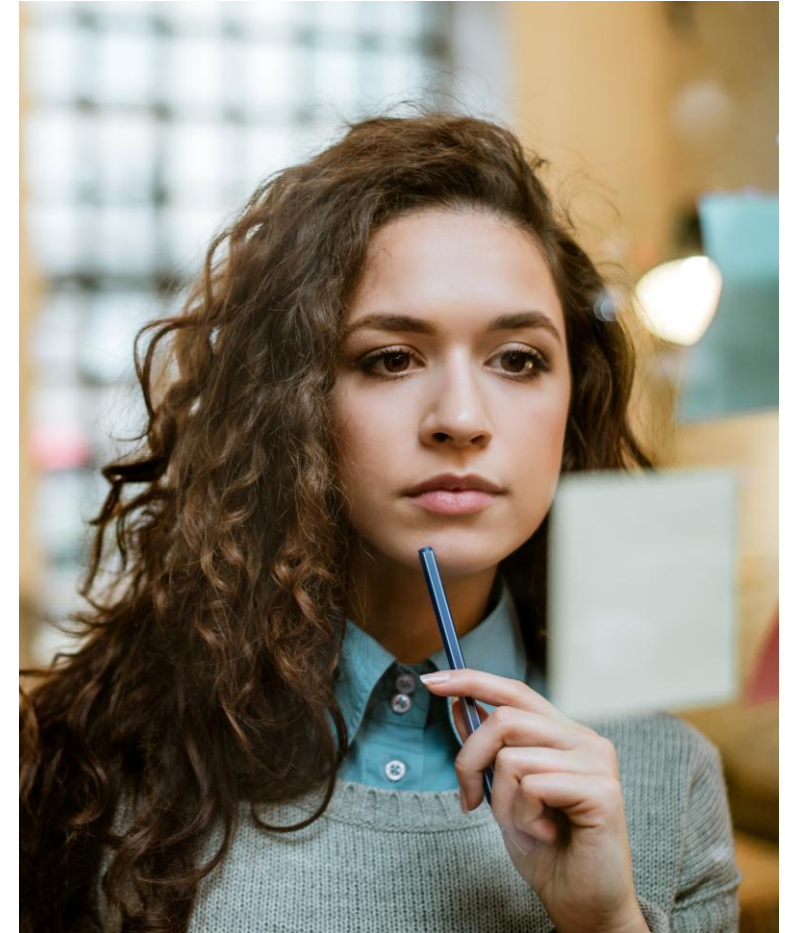
- S3 stores shared state
- Prevents local file dependency
- Supports team collaboration



TERRAFORM PLAN AND DRIFT DETECTION

Our lab will use terraform plan to identify drift.

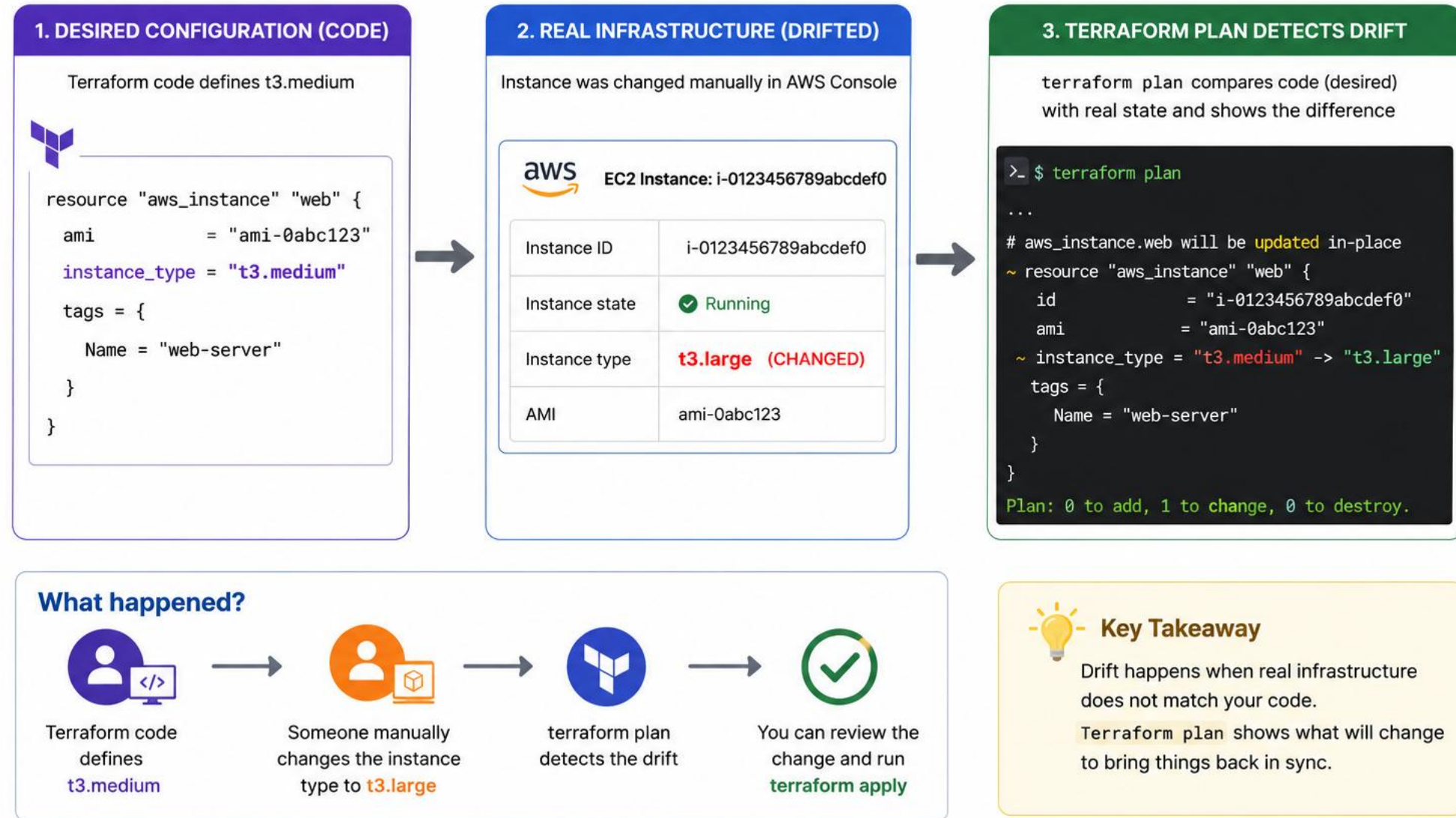
- Compares code, state, and live infrastructure
- Shows proposed changes before execution
- Safe way to detect configuration drift



Drift Example: EC2 Instance Type Changed Manually

Someone changed the EC2 instance type in the AWS Console from `t3.medium` to `t3.large`.

Terraform detects this drift when we run `terraform plan`.



POP QUIZ: Drift

What is configuration drift?

- A. When Terraform upgrades provider versions automatically
- B. When infrastructure no longer matches the desired configuration defined in code
- C. When multiple Terraform modules are combined
- D. When infrastructure is deployed to multiple regions



POP QUIZ: Drift

What is configuration drift?

- A. When Terraform upgrades provider versions automatically
- B. When infrastructure no longer matches the desired configuration defined in code**
- C. When multiple Terraform modules are combined
- D. When infrastructure is deployed to multiple regions



POP QUIZ: Drift

Which action commonly causes configuration drift?

- A. Running terraform apply
- B. Updating a Git repository
- C. Manually changing resources in the AWS Console
- D. Storing state files in S3



POP QUIZ: Drift

Which action commonly causes configuration drift?

- A. Running terraform apply
- B. Updating a Git repository
- C. Manually changing resources in the AWS Console**
- D. Storing state files in S3



POP QUIZ: Drift

What does terraform plan do?

- A. Deploys infrastructure automatically
- B. Deletes unused resources
- C. Compares desired configuration against current infrastructure and shows proposed changes
- D. Creates a new state file



POP QUIZ: Drift

What does terraform plan do?

A. Deploys infrastructure automatically

B. Deletes unused resources

C. Compares desired configuration against current infrastructure and shows proposed changes

D. Creates a new state file



LAB 3.3



CONGRATULATIONS

Modern monitoring platforms collect and correlate metrics, logs, and traces to provide visibility into applications and infrastructure. Solutions such as Dynatrace offer an integrated observability experience, while open-source alternatives like Grafana, Loki, Tempo, and Prometheus provide similar capabilities through a modular approach.

Infrastructure should also be monitored for configuration drift to ensure deployed systems remain aligned with their intended design.

